



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт (Филиал) № 8 «Компьютерные науки и прикладная математика»

Образовательный центр Института № 8

Группа М8О-408Б-22 Направление подготовки 01.03.02 «Прикладная математика и информатика»

Профиль Информатика

Квалификация: бакалавр

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

На тему: Система для сравнительного нагрузочного тестирования и визуального анализа производительности реляционных баз данных

Автор ВКРБ: Останина Анна Андреевна ()

Руководитель: Киндинова Виктория Валерьевна ()

Консультант: Воронцов Данила Артемьевич ()

Консультант: ()

Рецензент: ()

К защите допустить

Заведующий кафедрой № 806 Крылов Сергей Сергеевич ()

____ мая 2026 года

Москва 2026

РЕФЕРАТ

Выпускная квалификационная работа бакалавра состоит из 69 страниц, 7 рисунков, 6 таблиц, 12 использованных источников, 2 приложений.

НАГРУЗОЧНОЕ ТЕСТИРОВАНИЕ, РЕЛЯЦИОННЫЕ БАЗЫ ДАННЫХ, СУБД, ПРОИЗВОДИТЕЛЬНОСТЬ, СРАВНИТЕЛЬНЫЙ АНАЛИЗ, ВЕБ-ИНТЕРФЕЙС, МЕТРИКИ ПРОИЗВОДИТЕЛЬНОСТИ, ВИЗУАЛИЗАЦИЯ

Объектом исследования являются реляционные системы управления базами данных и их поведение под контролируемой нагрузкой. Предметом исследования выступают методы, алгоритмы и инструментальные средства для автоматизированного нагрузочного тестирования, сбора метрик производительности и визуализации результатов с целью сравнительного анализа. Объектом разработки является программный комплекс для сравнительного нагрузочного тестирования реляционных СУБД. Цель работы — создание инструмента, обеспечивающего автоматизированное тестирование, сбор метрик производительности и их анализ через единый веб-интерфейс.

В работе применены методы системного и объектно-ориентированного проектирования, математической статистики для сопоставления прогонов и экспериментального исследования производительности СУБД. Разработана клиент-серверная система с веб-интерфейсом и поддержкой MySQL, PostgreSQL и MariaDB; реализованы формирование нагрузки по сценариям, отображение метрик в реальном времени, хранение истории прогонов, сравнительный анализ результатов, а также подготовка и восстановление состояния баз данных при тестах с операциями записи. Развёртывание системы выполняется средствами Docker Compose.

Работоспособность подтверждена экспериментально на двух наборах данных — Sakila и Brazilian E-Commerce — при сравнении трёх СУБД; проведено 12 прогонов с нулевой долей ошибок.

Практическая значимость работы заключается в том, что результаты тестирования можно использовать для сравнения СУБД, оценки ускорения после изменений и выявления снижения производительности.

СОДЕРЖАНИЕ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ	5
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	8
ВВЕДЕНИЕ	9
1 АНАЛИТИЧЕСКАЯ ЧАСТЬ	11
1.1 Нагрузочное тестирование СУБД: понятия, цели, методологии .	11
1.2 Обзор существующих инструментов	14
1.2.1 Классические инструменты для реляционных СУБД	15
1.2.2 Коммерческие корпоративные решения	16
1.2.3 Воспроизводимость бенчмаркинга как концептуальный ориентир	17
1.3 Сравнительный анализ решений	17
1.4 Требования к разрабатываемой системе	19
1.5 Выводы по главе	21
2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ	22
2.1 Архитектура системы	22
2.2 Модуль генерации нагрузки и сбора метрик	24
2.2.1 Перечень собираемых метрик	25
2.2.2 Модуль управления индексами	27
2.2.3 Алгоритм сохранения и восстановления состояния базы данных	27
2.2.4 Механизм параметризации запросов	28
2.3 Проектирование слоя сценариев и групп баз данных	29
2.4 Подсистема визуализации и сравнительного анализа	30
2.5 Модель данных	31
2.6 Выводы по главе	33
3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ	35
3.1 Выбор и обоснование технологического стека	35
3.2 Реализация серверной части	36
3.2.1 Модуль генерации нагрузки	37
3.2.2 Модуль генерации сценариев	39
3.2.3 Callback-механизм потоковой передачи данных	40
3.2.4 Управление WebSocket-соединениями	40
3.2.5 Управление состоянием базы данных	41

3.2.6	Разрешение наборов сценариев нагрузки и запуск теста . . .	42
3.2.7	Модуль сравнительного анализа	43
3.2.8	Реализация статистических функций	44
3.2.9	Реализация REST API	45
3.2.10	Обработка ошибок	47
3.2.11	Слой доступа к данным	47
3.2.12	Управление подключениями к СУБД	48
3.3	Реализация клиентской части	49
3.3.1	Архитектура компонентов	49
3.3.2	Управление состоянием	50
3.3.3	Реализация WebSocket-хука	51
3.3.4	Модульный API-клиент	51
3.3.5	Визуализация данных	52
3.4	Контейнеризация и развёртывание	52
3.4.1	Инициализация и миграции	54
3.5	Выводы по главе	54
4	ТЕСТИРОВАНИЕ И АНАЛИЗ РЕЗУЛЬТАТОВ	56
4.1	Методика тестирования и тестовое окружение	56
4.2	Тестирование на наборе данных Sakila	59
4.2.1	Сравнение СУБД при смешанной нагрузке	59
4.2.2	Масштабируемость при росте нагрузки	60
4.3	Тестирование на наборе Brazilian E-Commerce	60
4.3.1	Сравнение СУБД при смешанной нагрузке	60
4.3.2	Масштабируемость при росте нагрузки	61
4.4	Транзакционная нагрузка и верификация резервного копирования	62
4.5	Сопоставление с внешним генератором нагрузки Sysbench . . .	62
4.6	Выводы по главе	63
	ЗАКЛЮЧЕНИЕ	64
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	66
	ПРИЛОЖЕНИЕ А Ссылка на GitHub	67
	ПРИЛОЖЕНИЕ Б Используемые датасеты	68

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящей выпускной квалификационной работе бакалавра применяют следующие термины с соответствующими определениями:

Верификация восстановления — проверка целостности базы данных после тестового прогона путём сравнения числа строк и, при необходимости, контрольных сумм таблиц с зафиксированными до теста значениями; подтверждает корректность автоматического восстановления состояния

Взаимная блокировка — ситуация при конкурентном доступе к данным, при которой два или более параллельных сеанса взаимно удерживают ресурсы, необходимые друг другу, что приводит к принудительному завершению одного из них СУБД и снижению общей пропускной способности

Виртуальный пользователь — эмулированная сущность, выполняющая операции нагрузки в ходе нагрузочного теста — одиночные SQL-запросы или атомарные транзакционные последовательности — для имитации реальной рабочей нагрузки; число виртуальных пользователей задаёт уровень конкурентности

Временной ряд — последовательность значений метрик производительности, зафиксированных через равные интервалы времени в ходе выполнения теста и используемых для построения графиков в реальном времени

Группа баз данных — именованная группа физических подключений к СУБД, объединённых единым профилем схемы; используется как единица конфигурации при одновременном нагрузочном тестировании нескольких экземпляров СУБД с одинаковой структурой данных

Задержка — время от отправки запроса к базе данных до получения ответа, измеряемое в миллисекундах

Коэффициент попаданий в кэш — доля запросов к базе данных, обработанных без обращения к дисковому хранилищу за счёт наличия данных в буферном пуле; служит индикатором эффективности использования оперативной памяти СУБД

Набор сценария нагрузки — конкретная конфигурация нагрузки для пары «профиль схемы + шаблон сценария»: перечень параметризованных SQL-запросов или транзакционных последовательностей операций изменения данных с весами выполнения и спецификацией индексов; фиксируется в неизменяемом снимке при запуске прогона

Нагрузочное тестирование — вид тестирования производительности, направленный на определение поведения системы под ожидаемой и повышенной нагрузкой

Процентиль — значение, ниже которого находится определённый процент измерений в наборе данных; например, р95 означает, что 95% запросов выполнены быстрее данного значения

Профиль схемы — описание структуры данных, общей для группы совместимых баз данных одной предметной области; служит основой для автоматической генерации наборов сценариев нагрузки

Прогрев — подготовительная фаза нагрузочного тестирования, состоящая из подфазы наращивания нагрузки и подфазы удержания на заданном уровне; метрики этой фазы не включаются в итоговый статистический анализ

Пропускная способность — количество успешно обработанных единиц нагрузки в единицу времени: запросов в секунду для сценариев с одиночными SQL-операциями или транзакций в секунду для транзакционных сценариев; в транзакционном режиме дополнительно фиксируется интенсивность отдельных SQL-шагов (запросов в секунду)

Размер эффекта — стандартизированная мера практической значимости различий между двумя выборками; в работе используется коэффициент Коэна d , интерпретируемый как пренебрежимый при $d < 0,2$, малый при $0,2 \leq d < 0,5$, средний при $0,5 \leq d < 0,8$ и большой при $d \geq 0,8$

Система управления базами данных — программное обеспечение для создания, ведения и управления базами данных, обеспечения доступа к ним и контроля целостности

Снимок состояния базы данных — фиксация состояния тестируемой базы данных перед прогоном с операциями записи: число строк в таблицах и, при включённой настройке, контрольные суммы данных; используется для последующей верификации восстановления

Сравнительный анализ — автоматизированное сопоставление результатов двух и более тестовых прогонов с вычислением описательной статистики, проверкой статистической значимости различий и формированием аналитического отчёта; поддерживает попарное сравнение СУБД при фиксированной нагрузке и серийный анализ при изменении числа виртуальных пользователей

Статистическая значимость — вероятностная оценка того, что наблюдаемое

различие метрик между сравниваемыми прогонами не обусловлено случайной вариацией; в системе определяется по p-value выбранного критерия с учётом поправки на множественные сравнения

Точка насыщения — уровень нагрузки, при превышении которого дальнейшее увеличение нагрузки не даёт прироста пропускной способности, а лишь увеличивает задержку

Транзакционный набор сценария нагрузки — набор сценария нагрузки для OLTP-шаблона, заданный вручную в виде последовательностей связанных операций изменения данных, выполняемых атомарно в рамках одной транзакции; в отличие от автогенерируемых наборов не формируется по схеме базы данных автоматически

Фаза измерения — основная фаза нагрузочного теста, следующая после завершения прогрева; метрики, собранные в измерительной фазе, используются для статистического анализа и сравнения прогонов

Хвостовая задержка — значение задержки на высоких перцентилях распределения p95 и p99, характеризующее поведение системы при обработке наиболее медленных запросов; является ключевым показателем стабильности производительности при конкурентной нагрузке

Шаблон сценария — логическое описание вида нагрузки без привязки к конкретной схеме данных; в системе предусмотрено шесть встроенных шаблонов: только чтение, только запись, смешанная лёгкая или тяжёлая нагрузка, OLTP и аналитические запросы; для пяти из них набор сценария нагрузки формируется автоматически по схеме, для OLTP — из встроенных транзакционных конфигураций

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящей выпускной квалификационной работе бакалавра применяют следующие сокращения и обозначения:

API — Application Programming Interface — программный интерфейс приложения

БД — база данных

CPU — Central Processing Unit — центральное процессорное устройство

CRUD — Create, Read, Update, Delete — создание, чтение, обновление, удаление

DML — Data Manipulation Language — язык манипулирования данными; операции INSERT, UPDATE и DELETE

СУБД — система управления базами данных

ER — Entity-Relationship — тип диаграммы модели данных «сущность — связь»

FDR — False Discovery Rate — уровень ложных открытий; поправка Бенджамини–Хохберга на множественные сравнения

FK — Foreign Key — внешний ключ

HTTP — Hypertext Transfer Protocol — протокол передачи гипертекста

I/O — Input/Output — ввод/вывод

JSON — JavaScript Object Notation — текстовый формат обмена данными

OLTP — Online Transaction Processing — оперативная обработка транзакций

ORM — Object-Relational Mapping — объектно-реляционное отображение

RAM — Random Access Memory — оперативная память

REST — Representational State Transfer — архитектурный стиль проектирования распределённых систем

SQL — Structured Query Language — язык структурированных запросов

TPC — Transaction Processing Performance Council — совет по производительности обработки транзакций

UUID — Universally Unique Identifier — универсальный уникальный идентификатор

VU — Virtual User — виртуальный пользователь

WebSocket — протокол полнодуплексной связи поверх TCP для передачи метрик в реальном времени

ВВЕДЕНИЕ

В современном цифровом мире, где данные стали одним из ключевых активов, эффективность их обработки и хранения напрямую определяет успешность бизнес-процессов, качество пользовательского опыта и, в конечном счёте, финансовые результаты организаций. Основой большинства информационных систем остаются системы управления базами данных, от производительности которых зависит скорость отклика приложений, стабильность работы под нагрузкой и возможность масштабирования. Особую значимость эти вопросы приобретают в контексте постоянного роста объёмов данных, усложнения запросов и повышения требований к доступности сервисов.

Однако существующие инструменты для нагрузочного тестирования баз данных зачастую не предоставляют комплексного решения. Многие из них требуют глубоких технических знаний для настройки, проводят тестирование в изолированной среде без возможности простого сопоставления результатов нескольких прогонов, а их выводы ограничиваются текстовыми отчётами или сырыми данными, требующими трудоемкой ручной обработки и анализа. Отсутствие интегрированной платформы, которая бы автоматизировала процесс тестирования, обеспечивала наглядную визуализацию ключевых метрик в реальном времени и позволяла проводить удобное сравнительное исследование, создаёт существенный барьер для оперативной диагностики узких мест и обоснованного принятия решений по оптимизации. В связи с этой проблемой становится актуальной разработка программного комплекса, который заполнит указанный пробел.

Целью данной выпускной квалификационной работы является разработка программного комплекса, позволяющего в автоматическом режиме проводить нагрузочное тестирование реляционных СУБД, собирать ключевые метрики производительности и предоставлять результаты в наглядном виде через единый веб-интерфейс для последующего анализа и сравнения.

Для достижения поставленной цели необходимо решить следующие задачи:

- 1) провести анализ современных подходов к нагрузочному тестированию СУБД и сформулировать требования к разрабатываемой системе;

- 2) разработать и реализовать ядро системы для автоматизированного тестирования производительности с поддержкой SQL-сценариев и транзакционной нагрузки;
- 3) спроектировать и создать веб-интерфейс для управления процессом тестирования и визуального анализа результатов;
- 4) выполнить комплексное тестирование системы на тестовых стендах и проанализировать полученные результаты.

Объектом исследования выступают реляционные системы управления базами данных и их поведение под контролируемой нагрузкой. Предметом исследования являются методы, алгоритмы и инструментальные средства для автоматизированного проведения нагрузочного тестирования, сбора метрик производительности и визуализации результатов с целью сравнительного анализа.

В работе применяются методы системного анализа для проектирования архитектуры, методы объектно-ориентированного проектирования для реализации модулей системы, методы математической статистики для сравнительного анализа результатов тестирования, а также методы экспериментального исследования для оценки производительности СУБД.

Практическая значимость работы заключается в создании инструмента, который позволяет не только измерять, но и наглядно сравнивать производительность различных СУБД, что является основой для обоснованного выбора технологий, оценки эффективности оптимизаций и предотвращения регрессии после внесения изменений. Целевой аудиторией системы являются инженеры по производительности, DevOps-инженеры, администраторы баз данных и разработчики.

Структура работы включает введение, четыре главы, заключение и список использованных источников. В первой главе проводится аналитический обзор существующих решений и формулируются требования. Во второй главе описывается проектирование архитектуры и модулей системы. В третьей главе рассматривается программная реализация. В четвёртой главе представлены результаты экспериментального тестирования — двенадцать прогонов — и анализ производительности на двух тестовых наборах данных.

1 АНАЛИТИЧЕСКАЯ ЧАСТЬ

1.1 Нагрузочное тестирование СУБД: понятия, цели, методологии

Производительность баз данных является одним из определяющих факторов качества современных информационных систем. С ростом объёмов обрабатываемых данных и увеличением числа одновременных пользователей задача объективной оценки производительности СУБД приобретает критическое значение. Нагрузочное тестирование представляет собой основной метод её решения, позволяя определить поведение системы под контролируемой нагрузкой и выявить потенциальные узкие места до их проявления в промышленной эксплуатации [1].

Нагрузочным тестированием называется вид тестирования производительности, направленный на определение поведения системы при ожидаемой и повышенной нагрузке, создаваемой путём эмуляции работы множества одновременных пользователей. Важно разграничивать нагрузочное тестирование и стресс-тестирование: первое оценивает работу системы в рамках проектных параметров нагрузки, второе определяет границы работоспособности при экстремальных значениях, превышающих нормальные эксплуатационные характеристики. Отдельным видом является тестирование стабильности, при котором система подвергается длительной нагрузке для выявления деградации производительности, утечек памяти и накопления фрагментации данных. Совместное применение указанных видов тестирования позволяет получить полную картину поведения СУБД в различных нагрузочных сценариях.

Ключевыми метриками, измеряемыми в процессе нагрузочного тестирования СУБД, являются задержка и пропускная способность [2]. Задержка характеризует время от завершения единицы нагрузки до получения ответа и измеряется в миллисекундах; для сценариев с одиночными SQL-операциями единицей служит запрос, для транзакционных сценариев — атомарная последовательность операций с фиксацией результата. Для полного описания распределения задержек применяются среднее значение, медиана (p50), процентиля p95 и p99, минимум, максимум и стандартное отклонение. Медиана отражает типичное время отклика, p95 и p99 характеризуют хвост распределения, а стандартное отклонение и размах (минимум–максимум)

позволяют оценить нестабильность. Использование исключительно среднего значения задержки является недостаточным, поскольку оно маскирует выбросы, способные существенно влиять на пользовательский опыт.

Пропускная способность измеряется числом успешно обработанных единиц нагрузки в секунду: запросов в секунду для SQL-сценариев и транзакций в секунду для транзакционных сценариев. Дополнительно фиксируется полная интенсивность завершённых операций в секунду, включая ошибочные, а в транзакционном режиме — вторичная интенсивность отдельных SQL-инструкций внутри транзакций (запросов в секунду), поскольку одна транзакция может включать несколько шагов. Зависимость между задержкой и пропускной способностью нелинейна: при достижении определённого порога нагрузки пропускная способность перестаёт расти, а задержка начинает экспоненциально увеличиваться. Точка, в которой пропускная способность достигает максимума, называется точкой насыщения и является важным параметром при определении предельной нагрузки на систему.

Помимо основных метрик, в процессе тестирования отслеживаются системные показатели хоста: загрузка центрального процессора (в процентах), использование оперативной памяти (в процентах и мегабайтах), интенсивность дискового ввода-вывода (операции в секунду и скорости чтения/записи), а также входящий и исходящий сетевой трафик. Из внутренних показателей СУБД особый интерес представляют коэффициент попаданий в кэш, число ожиданий блокировок, число взаимных блокировок и конфигурируемый размер буферного пула. Коэффициент попаданий в кэш отражает долю обращений, обслуженных без чтения с диска, и служит индикатором эффективности использования буферного пула. Ожидания блокировок и взаимные блокировки возникают при конкурентном доступе к данным и снижают общую пропускную способность; рост их частоты при увеличении числа параллельных транзакций сигнализирует о необходимости оптимизации структуры запросов или уровня изоляции транзакций.

Стандартизацией методологий тестирования производительности СУБД занимается организация TPC — всемирный консорциум, устанавливающий стандарты для оценки быстродействия и надёжности систем обработки транзакций и баз данных [3]. Наиболее значимыми спецификациями TPC для реляционных СУБД являются TPC-C, TPC-H и TPC-E.

Спецификация TPC-C моделирует среду обработки заказов, в которой множество пользователей одновременно выполняют транзакции нескольких типов: оформление заказов, регистрация платежей, проверка статусов и мониторинг складских запасов. Бенчмарк охватывает пять типов параллельных транзакций различной сложности, обеспечивая комплексную проверку компонентов СУБД. Основной метрикой TPC-C является количество новых заказов в минуту — $tpmC$. Данная спецификация широко применяется для оценки OLTP-систем и реализована в ряде инструментов тестирования, в частности в HammerDB под наименованием TPROC-C. Спецификация TPC-H ориентирована на системы поддержки принятия решений и представляет собой набор бизнес-ориентированных ad-hoc запросов и модификаций данных; её основная метрика — составной показатель производительности запросов в час $QphH@Size$; в инструментах тестирования данная нагрузка фигурирует под наименованием TPROC-H. Спецификация TPC-E представляет более современную OLTP-модель, воспроизводящую деятельность брокерской фирмы с асинхронными операциями и сложным профилем чтения и записи; метрика — количество транзакций в секунду. По сравнению с TPC-C, TPC-E обеспечивает более реалистичную нагрузку для современных онлайн-систем.

Параллельно с автоматизацией тестирования сформировалась концепция наблюдаемости, объединяющая сбор, хранение и анализ телеметрических данных в режиме реального времени. Ключевым компонентом такого стека в области мониторинга метрик является система Prometheus — открытое программное обеспечение для сбора и хранения временных рядов с многомерной моделью данных [4]. Данные в Prometheus идентифицируются по имени метрики и набору меток в формате ключ–значение, что позволяет выполнять гибкую фильтрацию и агрегацию с помощью языка запросов PromQL. Для унификации инструментирования приложений разработан стандарт OpenTelemetry — независимый от поставщика фреймворк, охватывающий трассировки, метрики и журналы и обеспечивающий передачу телеметрии через единый компонент OpenTelemetry Collector в различные бэкенды [5]. Совместное применение Prometheus и OpenTelemetry формирует полноценный стек наблюдаемости, позволяющий коррелировать результаты нагрузочных тестов с системными показателями. Необходимо отметить, что организация TPC расширяет область

стандартизации за рамки реляционных СУБД: в частности, спецификация TRCх-AI адресована платформам машинного обучения и аналитики данных, однако данное направление выходит за рамки предметной области настоящей работы.

Типовая методология нагрузочного тестирования СУБД включает несколько последовательных этапов. На этапе подготовки определяется цель тестирования, выбирается или формируется тестовая база данных, создаются сценарии нагрузки и настраивается тестовый стенд. На этапе прогрева система выводится на стабильный режим работы, исключая влияние начальной инициализации кэшей и буферов. Основной этап заключается в выполнении нагрузки с заданным числом виртуальных пользователей и фиксацией метрик через равные интервалы времени. Завершающий этап анализа предполагает вычисление агрегированных показателей, построение зависимостей метрик от нагрузки и формулировку выводов.

Таким образом, нагрузочное тестирование СУБД представляет собой комплексный процесс, охватывающий определение целей, выбор методологии, генерацию контролируемой нагрузки, сбор и анализ метрик производительности. Стандартизированные спецификации TRC обеспечивают сопоставимость результатов между различными системами. Рассмотренные методологии и метрики составляют теоретическую основу для анализа существующих инструментальных решений, представленного в следующем разделе.

1.2 Обзор существующих инструментов

Прежде чем анализировать конкретные инструменты, необходимо определить критерии их отбора и логику классификации. В рамках настоящего обзора рассматриваются решения, непосредственно применимые к нагрузочному тестированию реляционных СУБД: инструменты должны поддерживать хотя бы одну из целевых СУБД, генерировать контролируемую нагрузку и собирать метрики производительности. По характеру своего происхождения и целевой аудитории рассматриваемые решения разбиваются на два основных класса: классические инструменты для реляционных СУБД с открытым исходным кодом и коммерческие корпоративные решения. Отдельно анализируется концепция воспроизводимости бенчмаркинга на примере фреймворка Benchkit — она не относится к генераторам нагрузки, но

задаёт ориентир для централизованного хранения и сопоставления результатов. Такая структура обзора позволяет выявить не только достоинства и недостатки отдельных продуктов, но и закономерности, характерные для каждого класса в целом.

1.2.1 Классические инструменты для реляционных СУБД

Sysbench — кроссплатформенный инструмент нагрузочного тестирования и бенчмаркинга с открытым исходным кодом, изначально созданный для MySQL и впоследствии расширенный поддержкой PostgreSQL. Инструмент предоставляет набор встроенных OLTP-сценариев, охватывающих операции чтения, записи, обновления и удаления данных, и допускает создание пользовательских сценариев на языке Lua, что обеспечивает значительную гибкость конфигурирования. Ключевые метрики выходного отчёта — количество транзакций в секунду и время отклика. Основным достоинством Sysbench является сочетание простоты установки с широкими возможностями параметризации через Lua-скрипты. Вместе с тем инструмент ограничен интерфейсом командной строки, лишён механизмов хранения истории тестов и не предусматривает автоматизированного сравнения результатов между прогонами, что вынуждает пользователя самостоятельно интерпретировать и сопоставлять текстовые отчёты.

Утилита `pgbench` входит в стандартную поставку PostgreSQL и моделирует банковскую транзакционную нагрузку по мотивам спецификации TPC-B, измеряя количество транзакций в секунду при заданном числе параллельных клиентских соединений. Инструмент поддерживает выполнение пользовательских SQL-скриптов, что несколько расширяет область его применения за пределы встроенного бенчмарка. Главное достоинство `pgbench` состоит в отсутствии необходимости установки дополнительного программного обеспечения для пользователей PostgreSQL. Однако жёсткая привязка к одной СУБД, упрощённая модель нагрузки и минимальные возможности визуализации существенно сужают сферу применимости инструмента и делают его скорее отправной точкой для экспресс-оценки, нежели полноценным средством сравнительного анализа производительности.

HammerDB — инструмент нагрузочного тестирования и бенчмаркинга баз данных с открытым исходным кодом, разрабатываемый под эгидой

организации TPC. Актуальная версия 5.0 распространяется под лицензией GNU GPL v3.0 [6]. В данном выпуске HammerDB перешёл на бинарные дистрибутивы в виде единых исполняемых файлов для трёх режимов работы: графический интерфейс, интерфейс командной строки и веб-сервис. Ядро инструмента переработано на базе Tcl/Tk 9.0 — первого крупного обновления за более чем 27 лет. HammerDB поддерживает MariaDB, PostgreSQL, Oracle, SQL Server, MySQL и IBM Db2. Инструмент реализует рабочие нагрузки TPROC-C и TPROC-H — производные от спецификаций TPC-C и TPC-H — с результирующими метриками NOPM (New Orders Per Minute) и TPM (Transactions Per Minute). Ключевым преимуществом HammerDB является широкая поддержка СУБД в сочетании с соответствием отраслевым стандартам и доступностью официальных образов Docker для интеграции в CI/CD-окружения. Существенным ограничением остаётся отсутствие централизованного хранения истории тестов и автоматизированного сравнения прогонов.

1.2.2 Коммерческие корпоративные решения

Benchmark Factory 9.1 — коммерческий инструмент тестирования производительности баз данных, позиционируемый как многоплатформенное решение для корпоративного сегмента [7]. Benchmark Factory позволяет проводить воспроизведение рабочей нагрузки, отраслевые стандартизированные бенчмарки и тестирование масштабируемости. Инструмент доступен для Oracle, SQL Server, IBM DB2, SAP, MySQL, PostgreSQL, MariaDB и других баз данных через ODBC-подключение. Результаты тестирования сохраняются в централизованном репозитории с поддержкой встроенных отчётов и возможностью просмотра нескольких прогонов одновременно; предусмотрен REST API для автоматизации. Ключевое преимущество Benchmark Factory по сравнению с инструментами класса Oracle RAT — поддержка нескольких СУБД в рамках единого продукта с GUI. Вместе с тем инструмент является платным решением с графическим интерфейсом настольного приложения, что ограничивает его доступность для академических и небольших проектов и исключает нативный веб-доступ.

Oracle Real Application Testing — коммерческий инструмент корпорации Oracle для тестирования производительности баз данных Oracle, доступный исключительно в составе редакции Oracle Enterprise Edition. Компонент

Database Replay позволяет зафиксировать реальную рабочую нагрузку продуктивной базы данных и воспроизвести её на тестовой системе, а компонент SQL Performance Analyzer оценивает влияние изменений на выполнение отдельных SQL-запросов, сравнивая планы выполнения и метрики до и после изменений. Несмотря на высокую точность воспроизведения реальной нагрузки, Oracle RAT жёстко привязан к экосистеме Oracle и требует существенных лицензионных затрат, что принципиально ограничивает его применимость в контексте сравнительного тестирования нескольких СУБД.

1.2.3 Воспроизводимость бенчмаркинга как концептуальный ориентир

Отдельного упоминания заслуживает инструмент Benchkit — открытый фреймворк, решающий не задачу генерации нагрузки, а задачу воспроизводимости бенчмаркинга: декларативное описание окружения, фиксация всех параметров и результатов теста, автоматизация повторных запусков [8]. Данная концепция воспроизводимых конвейеров тестирования представляет интерес с точки зрения проектирования системы централизованного хранения и сравнения истории тестов.

Проведённый обзор показывает, что рассмотренные решения по-разному закрывают задачи генерации нагрузки и последующего анализа; их сопоставление по единым критериям приведено в следующем разделе.

1.3 Сравнительный анализ решений

Для систематизации результатов обзора проведён сравнительный анализ рассмотренных инструментов по критериям, непосредственно влияющим на удобство и эффективность практики нагрузочного тестирования СУБД. Выбор критериев обусловлен не техническими характеристиками самих инструментов, а практическими требованиями к процессу сравнительного тестирования нескольких СУБД. Пользовательский интерфейс рассматривается как критерий доступности: CLI-инструменты требуют технической экспертизы и не позволяют наблюдать динамику метрик в реальном времени, тогда как браузерный интерфейс снижает порог входа и обеспечивает немедленную визуальную обратную связь без установки

дополнительного программного обеспечения. Наличие истории тестов оценивается как критически важная характеристика: разовый прогон фиксирует состояние системы в единственный момент времени, тогда как только накопленная история позволяет выявлять регрессии производительности и долгосрочные тренды после изменений конфигурации или схемы данных. Возможность сравнения результатов производна от наличия истории и определяет, насколько процесс вынесения суждений о производительности автоматизирован, а насколько возложен на пользователя. Результаты анализа сведены в таблицу 1.

Таблица 1 – Сравнительный анализ инструментов нагрузочного тестирования СУБД

Критерий	HammerDB 5.0	Sysbench / pgbench	Oracle RAT	Benchmark Factory 9.1
Интер- фейс	GUI/CLI/Web	CLI	GUI/CLI	GUI/CLI/REST API
Поддер- живаемые СУБД	Oracle, SQL Server, Db2, MySQL, MariaDB, PostgreSQL	MySQL, PostgreSQL (pgbench — только PostgreSQL)	Только Oracle Database	Oracle, SQL Server, Db2, MySQL, PostgreSQL, SAP, MariaDB и др.
Хранение истории тестов	Отсутствует; только логи	Отсутствует	Частично	Централизо- ванный репози- торий
Сравнение тестов	Ручной анализ логов	Ручное	Встроенное	Встроенные отчёты
Визуали- зация	Простые графики	Текстовый отчёт	Детальные отчёты OEM	Встроенные отчёты
Модель распро- странения	Открытая	Открытая	Проприетарная	Проприетарная

Анализ таблицы 1 выявляет устойчивые закономерности, характерные для каждого класса инструментов, которые не сводятся к отдельным достоинствам и недостаткам конкретных продуктов.

Инструменты с открытым исходным кодом — HammerDB, Sysbench и pgbench — демонстрируют общий паттерн: высокая гибкость в части генерации нагрузки при принципиальном отсутствии механизмов долгосрочного накопления и сравнения результатов. Каждый прогон существует как изолированный артефакт; сопоставление результатов между прогонами остаётся ручной операцией, полностью возлагаемой на пользователя. Такая архитектура оправдана для разовых замеров производительности, однако делает перечисленные инструменты непригодными для систематического отслеживания изменений производительности во времени без создания вспомогательной инфраструктуры хранения и анализа данных.

Коммерческие продукты — Oracle RAT и Benchmark Factory 9.1 — предлагают наиболее развитый функционал в части анализа: централизованные репозитории результатов, встроенные сравнительные отчёты, поддержку нескольких типов рабочей нагрузки. Однако данный класс инструментов ограничен по-разному: Oracle RAT жёстко привязан к экосистеме одного поставщика и недоступен вне Oracle Enterprise Edition, тогда как Benchmark Factory, при широте поддерживаемых СУБД, является платным продуктом с настольным графическим интерфейсом, что исключает нативный веб-доступ и существенно ограничивает применимость в академических и некоммерческих проектах.

Таким образом, рассмотренные инструменты образуют фрагментированную экосистему, в которой ни одно решение с открытым исходным кодом не обеспечивает одновременно генерацию нагрузки на несколько реляционных СУБД, мониторинг метрик в реальном времени через браузерный интерфейс, централизованное хранение истории тестов и автоматизированное сравнение прогонов. Коммерческие продукты частично закрывают этот разрыв, однако за счёт закрытости экосистемы или высокой стоимости лицензирования. Выявленный функциональный разрыв определяет состав требований к разрабатываемой системе, сформулированных в следующем разделе.

1.4 Требования к разрабатываемой системе

На основании проведённого аналитического обзора и систематического выявления структурных недостатков существующих решений

сформулированы требования к разрабатываемой системе сравнительного нагрузочного тестирования реляционных СУБД. Требования разделены на функциональные, нефункциональные и ограничения; каждое из них непосредственно вытекает из функционального разрыва, зафиксированного в разделе 1.3.

Функциональные требования определяют действия, которые система обязана выполнять. Система должна обеспечивать поддержку трёх реляционных СУБД — MySQL, MariaDB и PostgreSQL — с возможностью сохранения нескольких конфигураций подключений и их совместного использования в рамках одного тестового прогона. Данное требование вытекает из необходимости обеспечить единую точку управления разнородными СУБД, что является ключевым пробелом существующих открытых инструментов.

Система должна поддерживать объединение нескольких физических подключений с одинаковой структурой данных в группу и автоматически подбирать к ней подходящий набор сценариев нагрузки, что позволяет проводить сравнительное тестирование без ручного конфигурирования каждого подключения. Генерация нагрузки должна осуществляться по параметризованным сценариям, которые задают состав операций — как отдельных SQL-запросов, так и последовательностей операций в рамках атомарных транзакций — с настраиваемым соотношением типов, числом виртуальных пользователей, числом итераций и длительностью прогрева.

Система должна обеспечивать потоковую передачу метрик производительности в веб-интерфейс с задержкой не более одной секунды. Система должна сохранять историю прогонов с возможностью фильтрации и просмотра ранее выполненных тестов. Система должна предоставлять инструменты сравнительного анализа двух и более прогонов — как попарного сравнения СУБД при одинаковой нагрузке, так и анализа изменения характеристик при последовательном увеличении числа виртуальных пользователей — с автоматическим применением статистических критериев, вычислением размера эффекта и формированием структурированного отчёта.

Система должна автоматически сохранять состояние тестируемых баз данных перед прогонами, включающими операции записи, и восстанавливать его по завершении теста с верификацией целостности данных, что обеспечивает воспроизводимость экспериментов.

Нефункциональные требования определяют качественные характеристики системы. Веб-интерфейс должен быть доступен через стандартный браузер без установки дополнительного программного обеспечения на стороне клиента — в противовес настольным GUI-решениям, характерным для коммерческого сегмента. Архитектура системы должна допускать расширение перечня поддерживаемых СУБД без значительной модификации кодовой базы, обеспечивая открытость к развитию. Развёртывание должно осуществляться с использованием контейнеризации, что гарантирует воспроизводимость тестового окружения и упрощает установку в различных инфраструктурах.

Ограничения фиксируют рамки применимости системы в текущей версии. Система предназначена для сравнительного анализа производительности, а не для проведения формальных сертифицированных бенчмарков ТРС. Генерация нагрузки осуществляется на одном сервере; горизонтальное масштабирование генераторов нагрузки в текущей версии не предусматривается. Область применения системы ограничена реляционными СУБД.

Сформулированные требования определяют функциональные и технические характеристики разрабатываемой системы и составляют непосредственную основу для проектирования её архитектуры, рассматриваемого во второй главе.

1.5 Выводы по главе

В первой главе обоснована актуальность нагрузочного тестирования СУБД, рассмотрены метрики и методологии, проведён обзор и сравнительный анализ существующих инструментов. Установлено, что открытые решения, как правило, сильны в генерации нагрузки, но не обеспечивают в одном продукте браузерный мониторинг в реальном времени, централизованную историю прогонов и автоматизированное статистическое сравнение; коммерческие средства частично восполняют этот разрыв ценой лицензирования или привязки к одной экосистеме. На этой основе сформулированы семь функциональных требований к разрабатываемой системе, нефункциональные требования и ограничения, которые в следующей главе положены в основу архитектуры и модели данных.

2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ

2.1 Архитектура системы

Разрабатываемая система построена по многоуровневой клиент-серверной архитектуре с разделением на серверную и клиентскую части. Серверная часть реализована на языке Python с использованием фреймворка FastAPI и обеспечивает генерацию нагрузки, сбор метрик, управление подключениями к тестируемым СУБД, хранение истории тестов, а также управление профилями схем и группами баз данных. Клиентская часть реализована на TypeScript с использованием фреймворка Next.js и предоставляет веб-интерфейс для управления тестированием, визуализации метрик в реальном времени и анализа результатов.

Взаимодействие между клиентской и серверной частями осуществляется по двум каналам: REST API — для CRUD-операций, запуска тестов и получения результатов; WebSocket — для трансляции метрик в реальном времени без периодического опроса сервера.

В процессе работы система взаимодействует с СУБД двух принципиально разных типов. Собственная СУБД системы — PostgreSQL 15, хранящая базу данных `project_data`, — предназначена для хранения истории тестов, наборов сценариев нагрузки, конфигураций подключений и результатов анализа. Вторая группа — целевые базы данных, работающие под управлением PostgreSQL, MySQL или MariaDB, над которыми непосредственно проводятся нагрузочные тесты. Такое структурное разделение позволяет изолировать служебные данные от тестируемых сред и исключить искажение результатов из-за операций сохранения метрик в ходе прогона.

Архитектура системы включает восемь серверных компонентов и клиентскую подсистему визуализации. Центральным серверным компонентом является модуль генерации нагрузки — ядро системы, реализующее параллельное выполнение SQL-запросов с заданным числом виртуальных пользователей, сбор метрик производительности и управление жизненным циклом теста. Модуль управления индексами сопровождает жизненный цикл индексов схемы в ходе прогона. Модуль управления подключениями предоставляет абстракцию подключения к различным СУБД, управление

пулами соединений и проверку доступности. Слой диалектов СУБД инкапсулирует сбор внутренних метрик и SQL-специфику PostgreSQL, MySQL и MariaDB. Модуль управления состоянием БД обеспечивает снимок и восстановление данных при write-сценариях и верификацию целостности. Модуль генерации сценариев вместе со слоем профилей схем и групп баз данных формирует каталог нагрузки и привязку подключений к схемам. Модуль хранения содержит репозитории для работы с базой данных истории, реализующие операции создания, чтения, обновления и удаления записей о тестах, результатах, сценариях и профилях. Модуль сравнительного анализа представляет собой сервис для сопоставления результатов нескольких тестовых прогонов, вычисления описательной статистики, применения статистических критериев и формирования аналитических отчётов. Подсистема визуализации реализована на стороне клиента в виде компонентов Next.js и отвечает за отображение временных рядов метрик, сравнительных диаграмм и аналитических отчётов на основе данных, получаемых от сервера.

Каждый компонент реализован в виде отдельного модуля с чётко определённым интерфейсом взаимодействия, что обеспечивает возможность независимой модификации и тестирования компонентов. Архитектура системы допускает расширение перечня поддерживаемых СУБД путём добавления новых драйверов подключения без модификации остальной кодовой базы.

Взаимодействие компонентов системы представлено на рисунке 1. Клиентский браузер отправляет HTTP-запрос на запуск теста, сервер создаёт запись в базе данных истории и запускает фоновую задачу. Фоновая задача вызывает LoadTester, который через DatabaseConnection подключается к тестируемому СУБД, выполняет SQL-запросы и собирает метрики. Метрики передаются через callback-механизм: класс TestStreamingCallback получает агрегаты от LoadTester, через ConnectionManager рассылает их подписчикам по WebSocket и сохраняет в базе данных истории через репозитории. Слой групп баз данных и профилей схем используется на этапе конфигурации: при выборе группы баз данных система автоматически разрешает привязанный профиль схемы и набор сценария нагрузки.

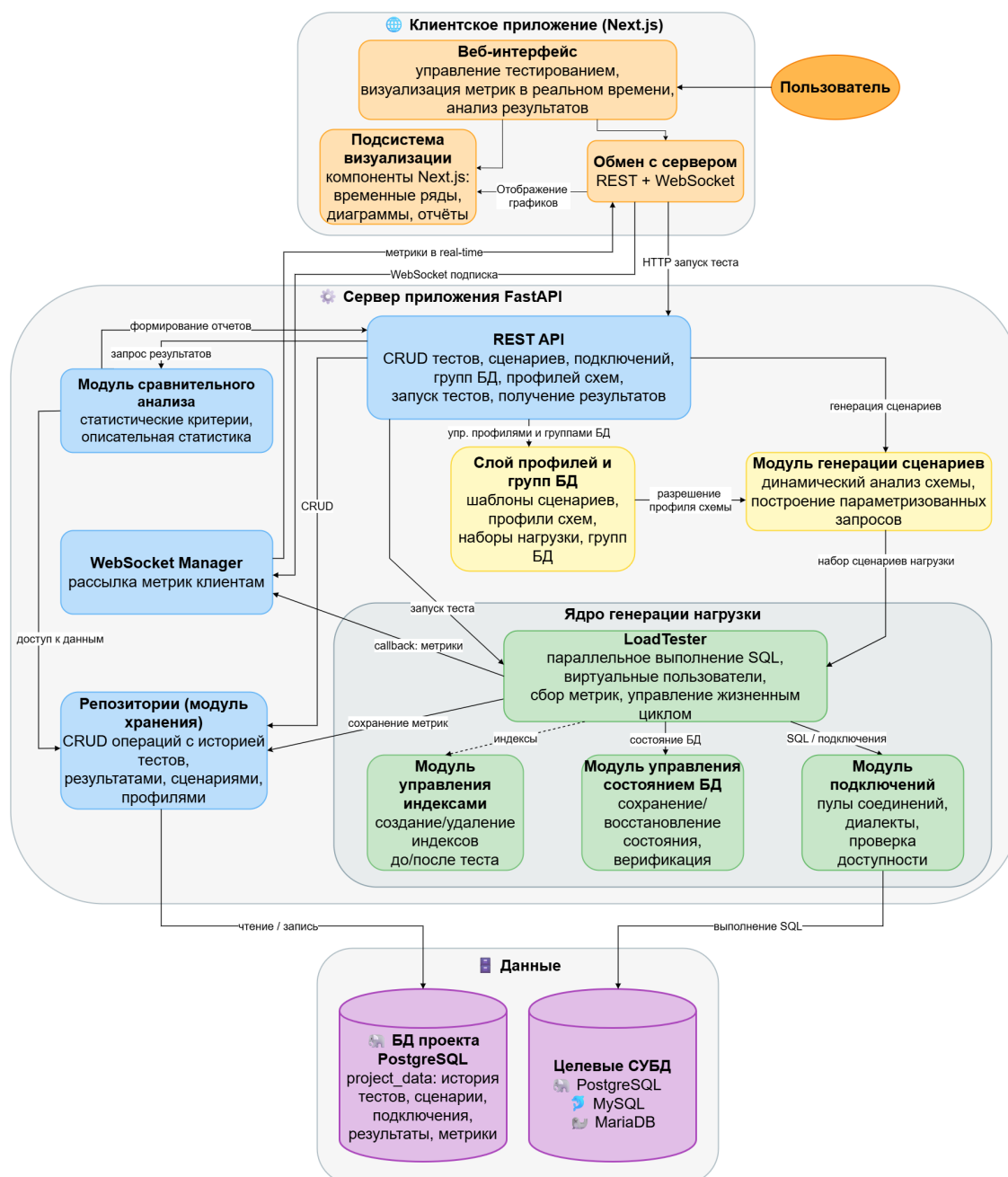


Рисунок 1 – Схема взаимодействия компонентов системы

2.2 Модуль генерации нагрузки и сбора метрик

При проектировании модуля генерации нагрузки определялись требования к параллельному выполнению SQL-запросов, точному замеру времени отклика, сбору системных и внутренних метрик СУБД, управлению индексами сценария, а также автоматическому управлению состоянием тестируемых баз данных.

Генерация нагрузки осуществляется с использованием асинхронной модели выполнения на базе библиотеки `asyncio`. Каждый виртуальный

пользователь реализуется как асинхронная задача через `asyncio.Task`, выполняющая заданное число итераций SQL-запросов. Параллельное выполнение задач обеспечивается функцией `asyncio.gather()`, которая запускает все задачи одновременно и ожидает их завершения. Такой подход позволяет эффективно использовать ресурсы сервера без создания избыточного числа потоков операционной системы.

Для каждой единицы нагрузки — одиночного SQL-запроса или пользовательской транзакции — фиксируется время отклика с использованием функции `time.perf_counter()`, обеспечивающей высокую точность измерений. Режим нагрузки задаётся полем `workload_mode` набора сценария: значение `query` соответствует одиночным операциям, `transaction` — атомарным транзакционным последовательностям. Результаты накапливаются в структуре данных, из которой по завершении измерительной фазы вычисляются итоговые метрики времени отклика: среднее, медиана (p50), процентиля p95 и p99, минимум, максимум, стандартное отклонение, а также показатели ошибок.

Сбор метрик производительности встроен в цикл работы виртуальных пользователей: при достижении интервала в одну секунду на каждом подключении агрегируются результаты за прошедшее окно и выполняются следующие действия:

- 1) вычисляются агрегированные метрики за интервал на основе накопленных результатов запросов или транзакций, включая успешную и полную пропускную способность;
- 2) через библиотеку `psutil` собираются системные метрики хоста;
- 3) через слой диалектов СУБД собираются внутренние метрики движка;
- 4) агрегат передаётся через `callback`-механизм для трансляции в реальном времени и сохранения в базе данных истории.

Параллельно агрегаты персистируются в сущностях `TimeSeries` и `MetricSample`; назначение сущностей раскрыто в подразделе о модели данных.

2.2.1 Перечень собираемых метрик

Состав фиксируемых показателей организован в четыре логические группы. Единица измерения времени отклика и основной пропускной

способности определяется полем `workload_mode` набора сценария нагрузки: при значении `query` учитывается одиночная SQL-операция, при `transaction` — атомарная пользовательская транзакция, включающая все SQL-шаги и завершение фиксацией или откатом. От этого же параметра зависят подписи метрик в итоговом отчёте прогона и в интерфейсе.

Первая группа охватывает время отклика в миллисекундах за измерительную фазу прогона. По завершении нагрузки вычисляются среднее значение, медиана `p50`, процентиля `p95` и `p99`, минимум, максимум и стандартное отклонение. В транзакционном режиме замер охватывает полный цикл выполнения транзакции, а не отдельные инструкции внутри неё.

Вторая группа описывает пропускную способность. Успешная пропускная способность в поле `throughput` выражает число успешно завершённых единиц нагрузки в секунду — запросов в секунду при сценариях с одиночными операциями или транзакций в секунду при транзакционной нагрузке. Полная интенсивность в поле `attempt_rate` фиксирует все завершённые единицы в секунду, включая завершившиеся с ошибкой. Для транзакционных сценариев дополнительно сохраняется вторичная интенсивность SQL-шагов в поле `qps`, а также число успешных и неуспешных транзакций, откатов, суммарное число SQL-инструкций и среднее число инструкций на одну транзакцию.

Третья группа включает системные метрики хоста, собираемые с интервалом в одну секунду через библиотеку `psutil`. Фиксируются загрузка центрального процессора в процентах, использование оперативной памяти в процентах и мегабайтах, интенсивность дисковых операций, скорости чтения и записи на диск в мегабайтах в секунду, а также входящий и исходящий сетевой трафик в тех же единицах. Итоговый снимок системных показателей сохраняется вместе с агрегированными результатами прогона.

Четвёртая группа объединяет внутренние метрики СУБД, получаемые через слой диалектов. За измерительную фазу накапливается изменение коэффициента попаданий в кэш, фиксируются размер буферного пула в мегабайтах с указанием соответствующего параметра конфигурации движка, число ожиданий блокировок и число взаимных блокировок, а также объёмы таблиц и базы данных в целом. Дополнительно учитываются частота ошибок и число активных соединений.

Метрики фазы прогрева маркируются отдельно и не передаются в

модуль сравнительного анализа. Секундные агрегаты записываются в сущность `TimeSeries` и используются для построения графиков в реальном времени; индивидуальные замеры задержки накапливаются в `MetricSample` и служат основой для статистических критериев при попарном сравнении прогонов.

2.2.2 Модуль управления индексами

Одним из проектных решений является управление индексами схемы в рамках жизненного цикла тестового прогона. До начала основного этапа тестирования модуль создаёт индексы базы данных в соответствии со спецификацией выбранного набора сценария нагрузки; после его завершения индексы удаляются, восстанавливая исходное состояние схемы. Такой подход позволяет оценивать влияние индексирования на производительность в рамках управляемого эксперимента: один и тот же набор SQL-запросов выполняется на схеме с заданной конфигурацией индексов, а результаты прогонов с разными конфигурациями становятся сопоставимыми. Каждый индекс в спецификации описывается набором атрибутов: имя таблицы, перечень столбцов, тип, признак уникальности и необязательное условие для частичных индексов.

2.2.3 Алгоритм сохранения и восстановления состояния базы данных

Тесты с операциями записи изменяют данные в тестируемой базе данных, что исключает повторяемость прогонов без предварительного сброса состояния. Для решения этой проблемы спроектирован модуль управления состоянием базы данных. Перед тестом анализируются SQL-запросы сценария: при наличии `write`-операций фиксируется снимок состояния базы данных, а после завершения теста данные восстанавливаются до исходного состояния. Для предотвращения конфликтов при параллельном тестировании нескольких баз данных одного типа предусмотрена блокировка на уровне типа СУБД, гарантирующая последовательное выполнение операций сохранения и восстановления.

В системе предусмотрены две взаимозаменяемые стратегии управления состоянием, выбор между которыми задаётся конфигурацией. Нативная

стратегия использует штатные CLI-утилиты конкретной СУБД для создания дампа и его последующего восстановления — этот подход обеспечивает максимальную точность и скорость операций. SQL-стратегия создаёт внутрибазовые копии таблиц средствами самой СУБД и не требует наличия внешних утилит, однако временно увеличивает объём данных в тестируемой базе. Если CLI-утилиты недоступны, система автоматически переключается с нативной стратегии на SQL-стратегию. После восстановления модуль верификации сравнивает число строк и контрольные суммы таблиц с зафиксированным до теста снимком и формирует отчёт о расхождениях при их обнаружении.

2.2.4 Механизм параметризации запросов

Для моделирования реалистичной нагрузки реализован механизм параметризации SQL-запросов. Каждый параметр запроса в наборе сценария нагрузки может быть сгенерирован одним из следующих способов:

- 1) `random_int` — случайное целое число из заданного диапазона от `min` до `max`;
- 2) `random_from_table` — случайное значение из указанного столбца существующей таблицы базы данных;
- 3) `sequential_int` — последовательное целое число, инкрементируемое на каждой итерации;
- 4) `uuid` — случайно сгенерированный UUID версии 4;
- 5) `fixed` — фиксированное значение, заданное в конфигурации параметра;
- 6) `random_string` — случайная строка заданной длины из алфавитно-цифровых символов;
- 7) `random_date` — случайная дата из заданного диапазона.

Такой подход позволяет варьировать данные в запросах и избегать эффектов кэширования планов выполнения, которые могут исказить результаты тестирования. Значения типа `random_from_table` кэшируются на уровне ключа `db_key:table:column` для снижения числа дополнительных запросов к БД в ходе теста.

Перед основным этапом тестирования выполняется фаза прогрева с линейным наращиванием нагрузки и удержанием на заданном уровне для инициализации кэшей и буферов СУБД. Метрики прогрева не входят в

итоговый анализ — учитывается только измерительная фаза, что обеспечивает сопоставимость прогонов.

2.3 Проектирование слоя сценариев и групп баз данных

Для обеспечения гибкого сопоставления нагрузочных сценариев с конкретными базами данных в системе спроектирован трёхуровневый слой управления сценариями. Данная архитектура позволяет описать логику нагрузки независимо от конкретной СУБД и автоматически подбирать конкретные SQL-запросы при выборе подключения.

Нижний уровень образует шаблон сценария — логическое описание вида нагрузки без привязки к конкретной схеме данных. Шаблоны определяют семантику нагрузки: в каталоге предусмотрено шесть встроенных шаблонов — только чтение, только запись, смешанная лёгкая нагрузка, смешанная тяжёлая нагрузка, OLTP и аналитические запросы. Встроенные шаблоны создаются автоматически при инициализации системы. Пять шаблонов, кроме OLTP, поддерживают автоматическую генерацию наборов сценариев нагрузки на основе анализа схемы — система извлекает метаданные таблиц и строит параметризованные SQL-запросы для каждого такого шаблона. Шаблон OLTP требует ручного определения набора транзакций: каждая операция задаётся явно в виде последовательности связанных инструкций изменения данных, что обеспечивает точное моделирование транзакционной нагрузки без автогенерации по схеме.

Средний уровень образует профиль схемы — описание структуры данных, общей для группы совместимых баз данных. Для подключения в поле `profile_source` фиксируется источник привязки: значение `auto` соответствует автоматическому определению, `inherited` — наследованию из группы, `manual` — ручному назначению, `pending_review` — ожиданию подтверждения; для группы баз данных в поле `profile_status` хранится статус профиля — черновик, подтверждён или несовместим. При подключении новой базы данных система автоматически анализирует её схему и предлагает наиболее подходящий профиль; коэффициент уверенности записывается в поле `profile_confidence`.

Верхний уровень образует набор сценария нагрузки — конкретная связка профиля схемы и шаблона сценария. В зависимости от типа шаблона конфигурация содержит перечень параметризованных SQL-запросов для

шаблонов чтения, записи, смешанных и аналитических сценариев либо набор транзакций с явно заданными последовательностями инструкций изменения данных для шаблона OLTP, а также спецификацию индексов. При запуске теста конфигурация фиксируется в неизменяемой снимке, который сохраняется вместе с прогоном, обеспечивая воспроизводимость результатов.

Группа баз данных объединяет набор физических подключений под единой моделью данных и связывает их с конкретным профилем схемы. Это позволяет тестировать несколько экземпляров одной и той же схемы — например, PostgreSQL, MySQL и MariaDB с одинаковой структурой данных — в рамках одного прогона без дополнительной настройки сценариев. Система отслеживает совместимость схем подключённых баз данных и формирует отчёт о расхождениях при их обнаружении.

Такой слой позволяет добавлять новые схемы через профиль и наборы сценариев без изменения ядра генерации нагрузки.

2.4 Подсистема визуализации и сравнительного анализа

Подсистема визуализации предназначена для представления результатов нагрузочного тестирования в наглядном виде и обеспечения возможности сравнительного анализа нескольких тестовых прогонов.

Модуль сравнительного анализа поддерживает два режима работы. Режим `per_test` предназначен для попарного сравнения СУБД в рамках одного прогона с несколькими подключениями: выполняется ранжирование СУБД по метрикам задержки и пропускной способности. Режим `series` ориентирован на серийный анализ от двух до пяти прогонов с различными уровнями нагрузки: строятся траектории изменения метрик и оцениваются характеристики масштабируемости — деградация хвостовых перцентилей, эластичность пропускной способности, точка насыщения; тренды проверяются критериями Спирмена и Манна–Кендалла. Режим задаёт пользователь на странице истории тестов и передаёт в запросе `POST /api/comparison/analyze` полем `analysis_mode`.

Сравнение выборок задержки выполняется с предварительной проверкой нормальности по критерию Шапиро–Уилка: при нормальном распределении применяется критерий Уэлча, при ненормальном — критерий Манна–Уитни. Такой подход обоснован характером распределения задержек: эмпирические распределения времени отклика, как правило, содержат

выбросы и не всегда близки к нормальным. Помимо статистической значимости, вычисляется размер эффекта — коэффициент Коэна d , — позволяющий разграничить статистически значимые, но практически пренебрежимые различия от различий, имеющих прикладную ценность. При попарном сравнении нескольких СУБД применяется поправка на множественные сравнения для контроля уровня ложных открытий.

Аналитический отчёт формируется по структурированным правилам. В режиме `per_test` он включает краткое резюме с высокоуровневым вердиктом, попарные сравнения, секцию практической значимости различий и конкретные рекомендации. В режиме `series` отчёт дополняется блоком `parameter_impacts` с анализом влияния различий конфигурации между прогонами; в обоих режимах автоматически выявляются характерные паттерны нестабильности производительности, деградации при масштабировании и высоких хвостовых задержек.

Для визуализации результатов сравнения предусмотрены три типа диаграмм: столбчатая диаграмма абсолютных значений задержки по среднему времени отклика и перцентилям $p95$ и $p99$, диаграмма размаха для оценки разброса и асимметрии распределений, а также временной график пропускной способности для выявления нестабильности и трендов.

2.5 Модель данных

Хранение данных системы организовано в базе данных PostgreSQL. Модель включает 15 сущностей, разделённых на три слоя: слой тестовых результатов, слой конфигураций подключений и групп баз данных, а также слой сценариев и профилей схем.

Слой тестовых результатов содержит четыре сущности. `TestRun` — запись о тестовом прогоне — хранит уникальный идентификатор, наименование, статус, конфигурацию теста в формате JSON: типы СУБД, число виртуальных пользователей, число итераций, время прогрева, идентификатор сценария и итоговую статистику, а также ссылку на группу баз данных и поля управления состоянием: признак наличия `write`-операций, статус и результат верификации восстановления. Сущность связана с остальными тремя через внешние ключи с каскадным удалением. `TestResult` хранит агрегированные метрики производительности для одной СУБД в рамках прогона в формате JSON: полный набор показателей времени отклика

(среднее, p50, p95, p99, минимум, максимум, стандартное отклонение), успешную и полную пропускную способность с учётом `workload_mode`, при транзакционной нагрузке — `qps` и счётчики транзакций, частоту ошибок, итоговый снимок системных метрик хоста и внутренних метрик СУБД. `TimeSeries` содержит секундные точки временного ряда: время отклика, успешную пропускную способность, загрузку CPU, использование RAM (проценты и Мбайт), дисковые IOPS и скорости чтения/записи, сетевой трафик — для графиков реального времени. `MetricSample` хранит сырые индивидуальные замеры задержки, необходимые для статистического анализа: в отличие от `TimeSeries`, каждая запись фиксирует отдельное наблюдение — замер SQL-запроса или транзакции; фаза прогрева и признак ошибки задаются полями `measurement_phase` и `is_error`.

Слой конфигураций подключений включает две сущности. `DatabaseConnectionConfig` — конфигурация подключения к СУБД — содержит тип СУБД, адрес сервера, порт, имя базы данных, зашифрованный пароль, ссылку на профиль схемы, а также поля `detected_profile_name`, `profile_confidence` и `profile_source`. `DatabaseGroup` объединяет несколько физических подключений под единой моделью данных: хранит ссылку на профиль схемы, ссылку на эталонное подключение, поля `profile_status` и `compatibility_status`, а также детальный отчёт о расхождениях схем.

Слой сценариев и профилей включает девять сущностей. `SchemaProfile` — профиль схемы данных — описывает структуру, общую для группы совместимых баз данных, и связывает подключения с наборами сценариев нагрузки. `ScenarioTemplate` задаёт логический шаблон вида нагрузки без привязки к схеме. `ScenarioBundle` объединяет профиль схемы и шаблон; уникальность активной пары «профиль + шаблон» обеспечивается частичным уникальным индексом. `ScenarioBundleQuery`, `ScenarioBundleParam` и `ScenarioBundleIndex` хранят SQL-запросы, параметры генерации и спецификации индексов сценария соответственно. Для поддержки транзакционной нагрузки OLTP-класса предусмотрены ещё три сущности: `ScenarioBundleTransaction` — заголовочная запись транзакции с именем и весом, `ScenarioBundleTransactionStep` — отдельная SQL-инструкция внутри транзакции с порядковым номером и описанием, `ScenarioBundleTransactionParam` — параметр инструкции транзакции с типом

генерации значения. Эти три сущности зеркально повторяют структуру сущностей для SQL-запросов применительно к транзакционной нагрузке и обеспечивают атомарное выполнение последовательностей операций изменения данных.

Все основные сущности используют UUID в качестве первичных ключей для обеспечения глобальной уникальности идентификаторов. Высоконагруженные таблицы TimeSeries и MetricSample используют целочисленный первичный ключ с автоинкрементом, что обеспечивает максимальную производительность вставки при потоковой записи метрик.

Модель данных представлена на ER-диаграмме на рисунке 2.

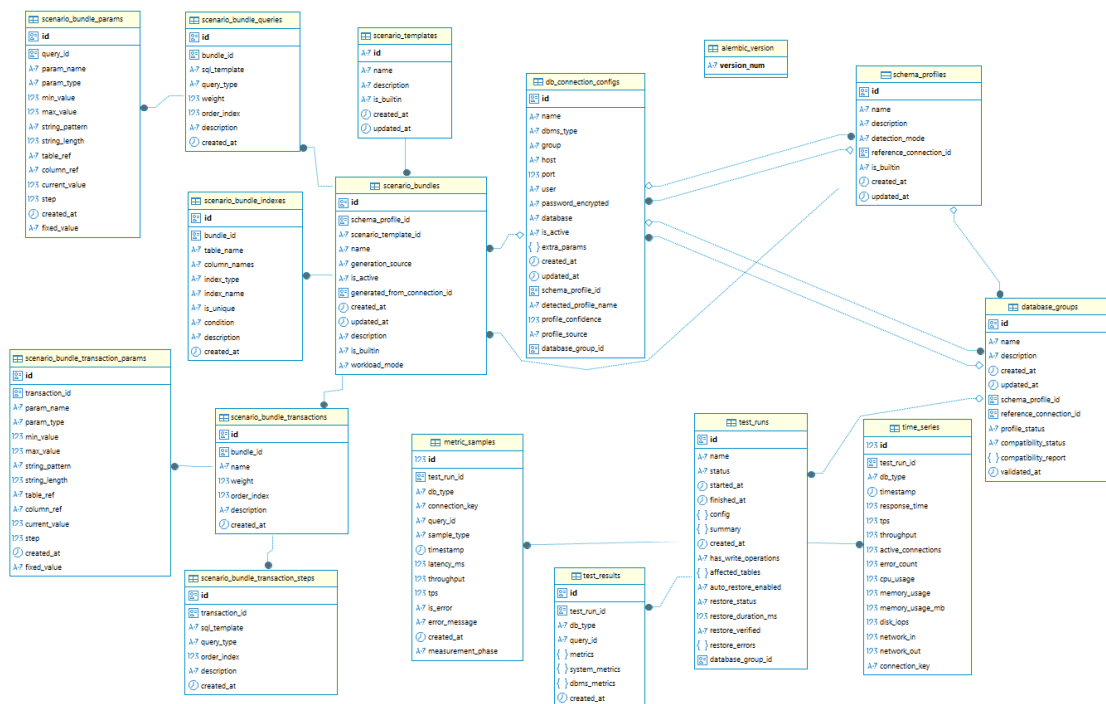


Рисунок 2 – ER-диаграмма модели данных системы

2.6 Выводы по главе

В настоящей главе спроектирована многоуровневая клиент-серверная архитектура с восемью серверными компонентами и подсистемой визуализации, разделением служебной `project_data` и целевых СУБД, а также потоком конфигурации, запуска теста по REST и доставки метрик по WebSocket. Детализированы генерация нагрузки и сбор метрик, управление индексами и состоянием БД, трёхуровневый каталог сценариев с группами баз данных, сравнительный анализ в режимах `per_test` и `series` и модель

данных из 15 сущностей в трёх слоях. Результаты проектирования составляют основу программной реализации, рассматриваемой в следующей главе.

3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

3.1 Выбор и обоснование технологического стека

На основе спроектированной в предыдущей главе архитектуры была выполнена программная реализация системы. Выбор технологического стека определялся требованиями к производительности, поддерживаемости и удобству разработки. Для серверной части был выбран язык Python версии 3.11 в сочетании с фреймворком FastAPI [9]. Python обеспечивает высокую скорость разработки благодаря развитой экосистеме библиотек для работы с базами данных, статистического анализа и визуализации. FastAPI предоставляет современный асинхронный веб-фреймворк с автоматической генерацией документации API через OpenAPI, встроенной валидацией данных через Pydantic версии 2 и высокой производительностью, сопоставимой с фреймворками на Node.js. Управление конфигурацией осуществляется через библиотеку pydantic-settings, загружающую параметры из переменных окружения и файла `.env`.

Для асинхронного выполнения запросов к СУБД выбраны драйверы `asyncpg` для PostgreSQL и `aiomysql` для MySQL и MariaDB. Оба драйвера обеспечивают неблокирующий ввод-вывод, что позволяет эффективно обрабатывать множество параллельных соединений в рамках одного потока операционной системы. В качестве ORM-слоя используется SQLAlchemy версии 2.0 с поддержкой асинхронных сессий, что обеспечивает типобезопасную работу с базой данных истории. Для управления миграциями схемы базы данных применяется инструмент Alembic, позволяющий описывать изменения схемы в виде версионизируемых Python-скриптов и применять их автоматически при старте приложения.

Для клиентской части выбран фреймворк Next.js версии 16 на базе React 19 с использованием TypeScript. Next.js обеспечивает клиентскую маршрутизацию через App Router и встроенную поддержку серверных компонентов. TypeScript гарантирует типобезопасность на всех уровнях клиентского приложения, снижая количество ошибок времени выполнения. Для управления состоянием приложения выбрана библиотека Zustand, отличающаяся минимальным объёмом кода и отсутствием необходимости в оборачивании приложения в провайдеры. Для визуализации данных

используются библиотеки `Recharts` и `Plotly`: `Recharts` предоставляет декларативный API для построения интерактивных графиков на базе SVG, `Plotly` применяется для диаграмм размаха и расширенных статистических визуализаций. Стилизация реализована через `Tailwind CSS` версии 4 с CSS-переменными как дизайн-токенами; компоненты интерфейса построены на базе `shadcn/ui` поверх `Radix UI`, что обеспечивает доступность и корректную работу с клавиатурой.

Для развёртывания всех компонентов системы используется `Docker Compose`; состав сервисов, порты и особенности сборки образов рассматриваются в разделе о контейнеризации настоящей главы.

3.2 Реализация серверной части

Серверная часть системы реализована в виде модульного приложения `FastAPI`. Точка входа находится в файле `main.py`, где инициализируется экземпляр приложения, регистрируются маршруты и создаётся `WebSocket`-эндпоинт.

Инициализация компонентов системы выполняется через модуль `initialize.py`, который создаёт экземпляры пяти репозиториев: `TestRepository`, `ConnectionRepository`, `ProfileRepository`, `ScenarioBundleRepository` и `DatabaseGroupRepository`. Каждый репозиторий принимает URL базы данных истории и создаёт собственный пул асинхронных соединений через `SQLAlchemy`. Настройки стратегии восстановления баз данных `settings.restore.*` загружаются из модуля конфигурации `core/config.py` и изменяются через `settings_routes` без отдельного репозитория. Такой подход реализует паттерн внедрения зависимостей и обеспечивает централизованное управление жизненным циклом компонентов. При недоступности базы данных инициализация не прерывает запуск приложения — репозитории устанавливаются в `None`, а соответствующая функциональность корректно деградирует.

Запуск и остановка приложения управляются через менеджер контекста `lifespan`. При запуске выполняются два последовательных шага:

- 1) автоматически применяются миграции `Alembic` командой `upgrade head`, что гарантирует актуальность схемы базы данных вне зависимости от версии развёртывания;
- 2) при успешной инициализации всех трёх ключевых репозиториев:

`connection_repository`, `profile_repository` и `scenario_bundle_repository` — запускается `LogicalScenarioBootstrap`: он создаёт шесть встроенных шаблонов сценариев и профили схем, а при наличии зарегистрированных подключений с определёнными профилями — генерирует SQL-наборы для пяти шаблонов с автогенерацией и встроенные OLTP-наборы для поддерживаемых профилей.

Первый шаг является fail-fast: при возникновении ошибки Alembic запуск приложения немедленно прерывается, что исключает работу системы с устаревшей схемой базы данных. Bootstrap на втором шаге выполняется условно — только при наличии рабочего подключения к базе данных проекта; если база данных недоступна, bootstrap пропускается без прерывания запуска. Повторный запуск обоих шагов не создаёт дублирующих записей — процедура полностью идемпотентна.

3.2.1 Модуль генерации нагрузки

Модуль генерации нагрузки реализован в классе `LoadTester` файла `load_tester/tester.py`. Основным методом запуска является `run_resolved_scenario_test_suite`, принимающий разрешённый набор сценария нагрузки и список идентификаторов подключений. В зависимости от типа набора метод выполняет либо одиночные SQL-запросы, либо транзакционные последовательности операций, где каждый виртуальный пользователь атомарно выполняет группу инструкций изменения данных. Единая структура каждого прогона: подготовка базы данных, двухфазный прогрев, измерительная фаза, сбор метрик, восстановление состояния, верификация. Блок `try/finally` гарантирует выполнение восстановления даже при возникновении ошибок в ходе тестирования.

Базовый метод `execute_query` выполняет отдельный SQL-запрос к указанной базе данных, замеряя время отклика через `time.perf_counter()`. На рисунке 3 приведён фрагмент реализации:

```

1  async def execute_query(self, db_key, query, query_id):
2      """ Выполнение одного запроса с измерением времени """
3      start_time = time.perf_counter()
4      error = None
5      rows_count = 0
6      db_type = self.db_connection.get_dbms_type(db_key)
7
8      try:
9          engine = await self.db_connection.get_engine_async(db_key)
10         async with engine.connect() as conn:
11             result = await conn.execute(text(query))
12             rows_count = len(result.fetchall()) if result.
returns_rows else 0
13             await conn.commit()
14         except Exception as e:
15             error = str(e)
16
17         end_time = time.perf_counter()
18         execution_time = (end_time - start_time) * 1000
19
20         return {
21             'query_id': query_id,
22             'execution_time_ms': execution_time,
23             'rows_count': rows_count,
24             'error': error,
25             'timestamp': datetime.now(timezone.utc).isoformat()
26         }

```

Рисунок 3 – Реализация метода `execute_query` класса `LoadTester`

Использование `time.perf_counter()` обеспечивает высокую точность замеров; асинхронное подключение через SQLAlchemy engine позволяет выполнять запросы без блокировки потока; результат включает не только время выполнения, но и количество обработанных строк и текст ошибки при её возникновении.

Каждый виртуальный пользователь реализован как асинхронная задача `asyncio.Task`. На каждой итерации выполняется одиночный запрос или транзакционная последовательность операций в соответствии с `workload_mode` разрешённого набора сценария; результаты накапливаются

в буфере. При достижении интервала в одну секунду вычисляются агрегированные метрики окна: среднее время отклика, успешная и полная пропускная способность, количество успешных и ошибочных операций; итог прогона дополняется перцентилями p50, p95, p99, минимумом, максимумом и стандартным отклонением за измерительную фазу. Данные передаются через callback-механизм для трансляции в WebSocket и сохранения в базу данных истории.

Для сбора системных метрик используется метод `get_system_metrics`, который через библиотеку `psutil` получает загрузку центрального процессора (проценты), использование оперативной памяти (проценты и мегабайты), дисковые операции в секунду, скорости чтения и записи (МиБ/с), а также входящий и исходящий сетевой трафик (МиБ/с). Метод `get_dbms_metrics` через слой диалектов в каталоге `database/dialects/` возвращает унифицированный набор внутренних показателей: коэффициент попаданий в кэш, размер буферного пула (мегабайты и метка параметра), число ожиданий блокировок и взаимных блокировок, объёмы таблиц. Для PostgreSQL используются представления `pg_stat_database` [10] и `pg_stat_bgwriter`, для MySQL и MariaDB — `performance_schema.data_lock_waits` и `SHOW GLOBAL STATUS`. Слой диалектов изолирует СУБД-специфичную логику и позволяет расширять поддержку СУБД без изменения основного цикла сбора метрик.

3.2.2 Модуль генерации сценариев

Модуль генерации сценариев реализован в классе `ScenarioGenerator` файла `database/scenario_generator.py`. Генератор принимает на вход подключение к целевой СУБД и тип шаблона сценария, анализирует метаданные схемы через `SchemaAnalyzer` и формирует структуру `ScenarioBundle` с массивами запросов и индексов.

Для пяти встроенных шаблонов с автогенерацией, кроме OLTP, `ScenarioGenerator` строит параметризованные SQL-шаблоны: шаблон только чтения включает выборки по первичному ключу и JOIN-запросы по внешним ключам, только записи — операции `INSERT` и `UPDATE` на изменяемых таблицах, смешанные шаблоны — комбинации `SELECT` и `UPDATE` с различными весами, аналитические запросы — агрегации, `JOIN` и диапазонные чтения по метаданным схемы. Шаблон OLTP не проходит через

генератор: для профилей Sakila и Brazilian E-Commerce наборы транзакций задаются встроенными конфигурациями. Параметры запросов типа `random_from_table` ссылаются на реальные столбцы схемы и кэшируются по ключу `db_key:table:column`. Класс `ScenarioBundleResolver` разрешает конкретный набор сценария нагрузки при запуске теста: по идентификатору набора или по паре «профиль + шаблон» он загружает запросы, параметры и индексы, формируя неизменяемый снимок конфигурации, который сохраняется вместе с прогоном для воспроизводимости.

3.2.3 Callback-механизм потоковой передачи данных

Ниже детализирован callback-механизм потоковой передачи метрик, упомянутый при описании `LoadTester`. Для связи с `WebSocket` и базой данных истории используется паттерн `Observer`. Класс `LoadTester` поддерживает два независимых callback: callback метрик, вызываемый на каждой итерации сбора агрегированных показателей, и callback статуса операций сохранения/восстановления состояния базы данных. Такое разделение позволяет клиенту независимо отслеживать ход теста и состояние данных.

Класс `TestStreamingCallback` файла `websocket_manager.py` выступает в роли наблюдателя: он получает метрики от `LoadTester` и транслирует их подписчикам. Метод формирует объект `TestMetricsUpdate`, содержащий все метрики производительности, системные показатели и прогресс выполнения. Данные передаются через `ConnectionManager` для рассылки всем `WebSocket`-подписчикам теста и одновременно сохраняются в базе данных истории как точка временного ряда. Для оптимизации записи сырых выборок метрик используется буферизация: накопление происходит до достижения размера буфера в 10 записей, после чего выполняется пакетная вставка через метод `add_metric_sample_batch`.

3.2.4 Управление WebSocket-соединениями

Управление `WebSocket`-соединениями реализовано в классе `ConnectionManager` файла `websocket_manager.py`. Класс хранит

активные соединения в словаре `active_connections`, ключом которого является идентификатор теста, а значением — список WebSocket-объектов. Метод `connect` принимает новое соединение и добавляет его в список, метод `disconnect` удаляет соединение и очищает пустые списки. Метод `broadcast_to_test` рассылает JSON-сообщение всем подписчикам конкретного теста. WebSocket-эндпоинт зарегистрирован по пути `/ws/test/{test_id}` и обрабатывает входящие ping-сообщения от клиента для поддержания соединения.

3.2.5 Управление состоянием базы данных

Оркестратор управления состоянием баз данных реализован в классе `DatabaseStateManager` файла `database/state_manager.py`. Класс управляет полным жизненным циклом операций: анализ запросов сценария, сохранение состояния базы данных, выполнение теста, восстановление исходных данных и верификация целостности.

Метод `prepare_for_test` сначала передаёт SQL-запросы сценария в модуль `QueryAnalyzer`, который определяет наличие write-операций `INSERT`, `UPDATE` и `DELETE`. Если сценарий содержит только `SELECT`-запросы, подготовка пропускается. При обнаружении write-операций определяются затронутые таблицы и полный список всех таблиц базы данных. Затем модуль `StateVerifier` фиксирует объект `StateFingerprint` — количество строк в каждой таблице, значения последовательностей и, при включённой настройке `verify_checksums`, контрольные суммы данных, — который впоследствии используется для верификации восстановления. После этого через настроенную стратегию сохраняется состояние всех таблиц базы данных: не только затронутых write-операциями, но и всех остальных, что гарантирует полный откат к исходному состоянию.

Нативная стратегия `NativeDumpStrategy` является основной: она вызывает CLI-утилиты конкретной СУБД — `pg_dump` для создания дампа и `pg_restore/psql` для восстановления у PostgreSQL; `mysqldump/mysql` для MySQL; `mariadb-dump/mariadb` для MariaDB, — сохраняя результат в виде файла дампа в директории `snapshots`. Для MariaDB используется самостоятельный клиент `mariadb-dump` с приоритетом перед `mysqldump`: это обеспечивает корректную обработку `DEFINER`-клауз и специфических

расширений синтаксиса. При восстановлении через нативную стратегию для MySQL/MariaDB дампы предварительно очищаются от DEFINER-клаузов, а для PostgreSQL выполняется TRUNCATE целевых таблиц перед data-only восстановлением через pg_restore/psql. Если CLI-утилиты не обнаружены в PATH, NativeDumpStrategy автоматически переключается на SqlBackupStrategy.

SQL-стратегия SqlBackupStrategy создаёт внутрибазовые копии таблиц через запросы вида CREATE TABLE ... AS SELECT * с фиксированным префиксом _loadtest_backup_ в той же схеме. При восстановлении таблицы очищаются через TRUNCATE и заполняются данными из копий через INSERT; порядок восстановления определяется топологической сортировкой по FK-зависимостям с временным отключением FK-ограничений. Данная стратегия не требует внешних утилит и работает через SQLAlchemy с любой поддерживаемой СУБД, однако временно увеличивает объём данных в тестируемой базе.

Метод restore_after_test выполняет восстановление под блокировкой asyncio.Lock, специфичной для типа СУБД, что исключает конфликты при параллельном тестировании. При включённой настройке verify_after_restore модуль StateVerifier строит новый StateFingerprint и сравнивает его с исходным: при полном совпадении числа строк и контрольных сумм восстановление считается успешным, при несовпадении формируется отчёт о расхождениях. По завершении операции внутрибазовые копии или временные файлы дампов удаляются.

3.2.6 Разрешение наборов сценариев нагрузки и запуск теста

Фоновая задача run_test_with_streaming создаёт экземпляр LoadTester, устанавливает callback для потоковой передачи метрик и callback статуса операций сохранения/восстановления состояния, разрешает набор сценария нагрузки через ScenarioBundleResolver и передаёт его в LoadTester вместе со списком идентификаторов подключений. Перед началом теста IndexManager создаёт индексы, описанные в ScenarioBundleIndex; по завершении теста индексы удаляются. По завершении результаты собираются, сохраняются в базе данных, а статус обновляется на completed или failed. При передаче параметра database_group_id тестовый прогон автоматически привязывается к

соответствующей группе баз данных.

3.2.7 Модуль сравнительного анализа

Модуль сравнительного анализа реализован в классе `ComparisonService` файла `comparison/service.py`. Конструктор класса принимает три репозитория: `repository` для загрузки данных тестов, `scenario_bundle_repository` для получения информации о наборах сценариев нагрузки и `connection_repository` для получения данных о подключениях. Основной метод `analyze` принимает объект `ComparisonRequest` с полями `analysis_mode`, `test_ids`, необязательным `baseline_id` и конфигурацией отчёта.

При обнаружении потенциальных проблем сопоставимости — разных наборов сценариев нагрузки, неравных временных периодов, смешанных типов СУБД — система формирует предупреждения, включаемые в итоговый отчёт.

Метод `analyze` маршрутизирует запрос по полю `analysis_mode`, задаваемому клиентом на странице истории тестов. В режиме `per_test` передаётся ровно один идентификатор прогона: загружаются его конфигурация, результаты и сырые выборки метрик по каждому подключению с ключом `connection_key`; для всех пар СУБД внутри прогона вычисляется описательная статистика задержки и пропускной способности, применяются статистические критерии с FDR-поправкой и формируется ранжирование; отчёт генерирует `PerTestReportGenerator`. В режиме `series` передаётся от двух до пяти идентификаторов прогонов с различными уровнями нагрузки: для каждой СУБД строятся траектории метрик, оцениваются показатели масштабируемости `stability_index`, `elasticity` и точка насыщения, тренды проверяются критериями Спирмена и Манна–Кендалла; базовый прогон задаётся полем `baseline_id`; отчёт генерирует `SeriesReportGenerator`. Оба генератора реализованы в модуле `analysis/report_generator.py`.

Сбор сырых выборок метрик реализован с трёхуровневым механизмом `fallback`: при наличии записей `MetricSample` используются они; при их отсутствии — данные из `TimeSeries`; в крайнем случае — агрегированные метрики из `TestResult` с соответствующим предупреждением о сниженной надёжности анализа. Это обеспечивает работоспособность сравнения даже

для тестов, выполненных до добавления модуля сбора сырых метрик.

Отчёт в режиме `per_test` включает краткое резюме с лидером по пропускной способности и СУБД с наименьшей средней задержкой, попарные сравнения с интерпретацией и ранжирование подключений. Отчёт в режиме `series` дополняется блоком `parameter_impacts` с анализом влияния различий конфигурации между прогонами; в обоих режимах автоматически выявляются паттерны нестабильности при высоком CV, деградации при масштабировании и высоких хвостовых задержек при соотношении p99 к медиане.

3.2.8 Реализация статистических функций

Модуль `comparison/statistics.py` содержит математический аппарат для статистического сравнения выборок. Функция `calculate_descriptive_stats` вычисляет описательную статистику через `numpy`: количество, среднее, медиану, стандартное отклонение, минимум, максимум, процентиля `p50`, `p95`, `p99`, коэффициент вариации CV, а также межквартильный размах IQR. Функция `check_normality` проверяет нормальность распределения через критерий Шапиро–Уилка с ограничением выборки в 5000 элементов.

Функция `compare_two_samples` реализует двухэтапный алгоритм выбора критерия. На первом этапе проверяется размер выборки: если количество элементов менее 10, статистический тест не выполняется и выводится предупреждение о недостаточном объёме данных; если размер выборки превышает 5000 элементов, из неё извлекается случайная подвыборка для снижения вычислительной нагрузки. Затем к полученной выборке применяется критерий Шапиро–Уилка: если *p*-значение превышает 0,05, нулевая гипотеза о нормальности не отвергается. На втором этапе по результату проверки нормальности выбирается критерий сравнения: при нормальном распределении применяется критерий Уэлча, не требующий равенства дисперсий; при ненормальном — критерий Манна–Уитни, являющийся непараметрическим аналогом *t*-критерия. Если *p*-значение выбранного критерия оказывается меньше 0,05, различие признаётся статистически значимым и формируется текстовая интерпретация с указанием процентного отклонения. При сравнении нескольких пар применяется FDR-поправка Бенджамини–Хохберга для контроля уровня ложных открытий.

3.2.9 Реализация REST API

REST API системы разделено на восемь модулей маршрутов, каждый из которых отвечает за свою область функциональности. Основные эндпоинты представлены в таблице 2.

Таблица 2 – Основные эндпоинты REST API

Модуль	Метод и путь	Назначение
test	POST /test/async	Асинхронный запуск теста
test	GET /test/async/{id}	Получение статуса теста
test	GET /test/async/{id}/results	Получение результатов
history	GET /history/tests	Список тестов с пагинацией
history	GET /history/tests/{id}	Детали конкретного теста
history	DELETE /history/tests/{id}	Удаление теста
connection	GET /api/connections	Список подключений
connection	POST /api/connections	Создание подключения
connection	POST /api/connections/test	Тестирование подключения
database_state	POST /.../{id}/backup	Сохранение состояния базы данных
database_state	POST /.../{id}/restore	Восстановление состояния базы данных
database_state	GET /.../{id}/estimate	Оценка объема данных базы
comparison	POST /api/comparison/analyze	Сравнительный анализ
schema-profiles	GET /api/schema-profiles	Список профилей схемы

Продолжение таблицы 2

schema-profiles	POST /api/schema-profiles	Создание профиля
schema-profiles	GET /.../{id}/bundles	наборы сценариев нагрузки профиля
schema-profiles	POST /.../{id}/bundles/generate	Генерация набора сценария нагрузки для профиля схемы
database-groups	GET /api/database-groups	Список групп баз данных
database-groups	POST /api/database-groups	Создание группы баз данных
database-groups	PUT /.../{id}/profile	Привязка профиля схемы к группе
database-groups	POST /.../{id}/bundles/generate	Генерация набора сценария нагрузки для профиля, привязанного к группе баз данных
database-groups	POST /.../{id}/connections/{connection_id}/confirm-profile	Подтверждение профиля схемы для подключения в группе
database-groups	PUT /.../{id}/reference-connection	Обновление эталонного подключения
settings	GET /api/settings/restore	Настройки восстановления
settings	PUT /api/settings/restore	Обновление настроек восстановления

Эндпоинты POST /.../{id}/bundles/generate в модулях schema-profiles и database-groups вызывают одну операцию генерации наборов сценариев нагрузки через ScenarioGenerator; различаются контекст запроса и проверки входных параметров.

Эндпоинт POST /test/async принимает параметры нагрузки,

создаёт запись прогона со статусом `pending`, запускает фоновую задачу `run_test_with_streaming` через `BackgroundTasks` `FastAPI` и возвращает клиенту `test_id` с `websocket_url` для подписки на метрики. Настройки стратегии восстановления по пути `/api/settings/restore` изменяются в памяти процесса до рестарта бэкенда.

3.2.10 Обработка ошибок

Обработка ошибок в серверной части реализована на нескольких уровнях. На уровне API маршрутов все исключения перехватываются и преобразуются в HTTP-статусы: `ValueError` — в 400 `Bad Request`, неожиданные исключения — в 500 `Internal Server Error`. На уровне бизнес-логики ошибки логируются с указанием контекста и, где возможно, обрабатываются без прерывания выполнения теста. Блок `try/finally` в методах тестирования гарантирует выполнение восстановления базы данных даже при критических ошибках.

При возникновении ошибки выполнения SQL-запроса метод `execute_query` не прерывает цикл итераций, а сохраняет текст ошибки в результате и продолжает выполнение следующих запросов. Это позволяет получить полную картину производительности, включая частоту ошибок, которая сама по себе является важной метрикой.

3.2.11 Слой доступа к данным

Слой доступа к данным реализован через паттерн `Repository`. Базовый класс `BaseRepository` обеспечивает асинхронное подключение к базе данных истории через `SQLAlchemy` с управлением сессиями через контекстный менеджер. Наследники реализуют специфичные операции для своих сущностей: `TestRepository` управляет тестовыми прогонами и временными рядами, `ConnectionRepository` — конфигурациями подключений с расшифровкой паролей, `ProfileRepository` — профилями схем и шаблонами сценариев, `ScenarioBundleRepository` — наборами сценариев нагрузки с их запросами и индексами, `DatabaseGroupRepository` — группами баз данных.

Метод `TestRepository.add_metric_sample_batch` выполняет

пакетное сохранение сырых выборок метрик. Код метода приведён на рисунке 4:

```
1 async def add_metric_sample_batch(  
2     self, test_run_id: str, samples: List[Dict]  
3 ):  
4     """ Пакетное сохранение сырых метрик """  
5     if not samples:  
6         return  
7  
8     async with self.get_session() as session:  
9         records = [  
10             MetricSample(  
11                 test_run_id=test_run_id,  
12                 db_type=s['db_type'],  
13                 connection_key=s['connection_key'],  
14                 query_id=s.get('query_id'),  
15                 sample_type=s.get('sample_type'),  
16                 timestamp=s['timestamp'],  
17                 latency_ms=s.get('latency_ms'),  
18                 throughput=s.get('throughput'),  
19                 is_error=s.get('is_error', False),  
20                 error_message=s.get('error_message'),  
21             )  
22             for s in samples  
23         ]  
24         session.add_all(records)  
25         await session.commit()
```

Рисунок 4 – Метод addMetricSampleBatch класса
TestRepository

Пакетная вставка снижает количество обращений к базе данных при высокой интенсивности тестирования. Каждая запись содержит тип выборки, что позволяет гибко выбирать уровень детализации данных при сравнительном анализе.

3.2.12 Управление подключениями к СУБД

Класс DatabaseConnection файла database/connection.py управляет подключениями к тестируемым СУБД. Подключения хранятся в

репозитории базы данных истории и загружаются лениво при первом асинхронном вызове.

Метод `_resolve_connection_key` обеспечивает поддержку shorthand-ключей: если передан ключ `mysql` или `postgresql` и существует ровно одно активное подключение данного типа, оно используется автоматически. При наличии нескольких подключений одного типа требуется явное указание `connection_id`. Кэширование конфигураций подключений в памяти исключает повторные обращения к базе данных при каждом запросе.

Для шифрования паролей подключений используется модуль `core/crypto.py`, реализующий симметричное шифрование в формате Fernet. Пароль шифруется перед сохранением в базе данных и расшифровывается при установлении подключения к тестируемой СУБД. Ключ шифрования задаётся через переменную окружения `ENCRYPTION_KEY`.

3.3 Реализация клиентской части

Клиентская часть системы реализована как одностраничное приложение на базе Next.js 16 с клиентской маршрутизацией. Главная страница в файле `app/page.tsx` определяет компонент-маршрутизатор, который условно отображает один из восьми компонентов-страниц в зависимости от значения `currentPage` в Zustand store. Общая компоновка включает верхнюю панель с логотипом и навигацией, боковое меню с переключением страниц и основную область контента.

3.3.1 Архитектура компонентов

Страница `HomePage` является стартовой и предоставляет краткое описание системы с кнопками быстрого перехода к конфигурации теста или истории. `ConnectionsPage` обеспечивает полный CRUD для сохранённых подключений к СУБД: добавление, редактирование, удаление, проверка доступности. При создании подключения страница отображает результат автоопределения профиля схемы с коэффициентом уверенности.

Страница `ConfigPage` является центральным узлом конфигурации теста. Она содержит несколько карточек: выбор группы баз данных или отдельных подключений, выбор режима теста, выбор сценария или набор

сценария нагрузки, настройка параметров нагрузки: числа виртуальных пользователей, числа итераций и времени прогрева, а также сводка конфигурации и кнопка запуска.

Страница `ScenariosPage` предоставляет браузер наборов сценариев нагрузки через компонент `logical-scenarios-browser`. В браузере отображаются группы баз данных, профили схем и связанные с ними наборы сценариев нагрузки с их SQL-запросами, параметрами и спецификацией индексов. Предусмотрены операции создания, редактирования, клонирования и генерации наборов сценариев нагрузки из эталонного подключения. Для каждой группы баз данных отображаются статусы `profile_status` и `compatibility_status`, что позволяет пользователю контролировать состояние привязки профиля и совместимость подключённых баз данных.

Страница `DashboardsPage` предоставляет дашборд реального времени с четырьмя вкладками: время отклика и пропускная способность, внутренние метрики СУБД, объёмы данных и системные метрики хоста. Каждая вкладка отображает секундные временные ряды для всех подключений группы одновременно.

Страница `HistoryPage` отображает таблицу выполненных тестов с возможностью выбора одного теста для режима `per_test` или от двух до пяти — для режима `series`. Страница `ComparisonPage` визуализирует ответ `POST /api/comparison/analyze`: резюме, таблицу метрик — в режиме `per_test` отклонения относительно первого подключения в порядке `connection_ids` прогона, в режиме `series` — относительно базового прогона, — диаграммы, статистические тесты, аналитический отчёт и при `series` блок `parameter_impacts`; экспорт результата в JSON выполняется кнопкой `Download` на той же странице. Страница `DatabaseStatePage` предоставляет панель ручного сохранения и восстановления состояния подключённых баз данных и оценки объёма данных.

3.3.2 Управление состоянием

Состояние приложения хранится в едином `store`, реализованном через библиотеку `Zustand`. Файл `lib/store.ts` определяет интерфейс `AppState` и создаёт `store` через `create<AppState>`. `Store` включает следующие группы данных: навигация — текущая страница и состояние боковой панели;

конфигурация теста — режим, сценарий, идентификатор набора сценария нагрузки или шаблона, признак использования индексов, число виртуальных пользователей, итерации и время прогрева; текущий тест; история тестов; данные реального времени — временные ряды для каждого подключения; словари имён и типов подключений; параметры сравнительного анализа.

Метод `addRealtimeData` обеспечивает добавление точек временного ряда с ограничением буфера.

Метод `.slice(-100)` ограничивает массив последними 100 точками для каждого подключения, что предотвращает чрезмерное потребление памяти при длительных тестах и обеспечивает плавную отрисовку графиков. Ключом в словаре `realtimeData` выступает уникальный идентификатор подключения, что позволяет отображать метрики для нескольких экземпляров одного типа СУБД независимо.

3.3.3 Реализация WebSocket-хука

Хук `useTestWebSocket` файла `hooks/use-test-websocket.ts` обеспечивает двустороннюю связь с сервером для получения метрик в реальном времени. При монтировании компонента с валидным `testId` автоматически устанавливается WebSocket-соединение по адресу `/ws/test/{testId}`. Каждые 30 секунд отправляется ping-сообщение для предотвращения разрыва соединения из-за таймаута.

При разрыве соединения во время выполнения теста производится автоматическое переподключение с интервалом 3 секунды. Переподключение выполняется только при статусе `running`, что предотвращает бесконечные попытки подключения после завершения теста. При получении сообщения типа `metrics` данные преобразуются в `TimeSeriesPoint` и добавляются в `store` через `addRealtimeData`, что вызывает реактивное обновление компонентов дашборда.

3.3.4 Модульный API-клиент

API-клиент реализован как класс `ApiClient` файла `lib/api/client.ts`. Базовый URL определяется переменной окружения `NEXT_PUBLIC_API_URL`. Метод `request<T>` централизует обработку ошибок: при получении HTTP-статуса, отличного от 2xx, из ответа

извлекается сообщение об ошибке и выбрасывается исключение `Error`. Все методы клиента типизированы через дженерики `TypeScript`, что обеспечивает проверку корректности данных на этапе компиляции.

Класс включает доменные методы для каждой группы эндпоинтов: методы управления подключениями и их профилями, методы работы с профилями схемы, методы управления группами баз данных, а также методы запуска тестов, получения истории и запуска сравнительного анализа. Экземпляр `ApiClient` экспортируется как `singleton`; компоненты импортируют методы через barrel-экспорт `lib/api.ts`.

3.3.5 Визуализация данных

Визуализация метрик реализована с использованием библиотек `Recharts` и `Plotly`. Секундные ряды дашборда отображаются в переиспользуемом компоненте `TimeSeriesChart`; подписи пропускной способности зависят от `workload_mode` прогона. На странице сравнительного анализа для диаграмм размаха и расширенных статистических графиков применяется `Plotly`.

Стилизация интерфейса выполнена через `Tailwind CSS v4` с использованием дизайн-токенов для цветов, отступов и размеров шрифтов; поддерживается тёмная тема через `className="dark"` на корневом элементе. Компоненты интерфейса построены на базе `Radix UI` через библиотеку `shadcn/ui`. Уведомления пользователю отображаются через библиотеку `Sonner` в виде всплывающих сообщений.

3.4 Контейнеризация и развёртывание

Развёртывание системы организовано через `Docker Compose`. Основной файл `docker-compose.yml` определяет три сервиса и представлен в таблице 3.

Таблица 3 – Сервисы Docker Compose

Сервис	Образ/источник	Назначение	Порт
project-db	postgres:15-alpine	База данных проекта	5433:5432
backend	Dockerfile на Python	FastAPI REST API и WebSocket-сервер	8000:8000
frontend	Dockerfile на Node.js	Next.js веб-интерфейс	3000:3000

Сервис `project-db` развёртывается на базе образа `postgres:15-alpine`. Сервис настроен с именованным томом `project_db_data` для персистентности данных, `healthcheck` через `pg_isready` с интервалом 5 секунд и политикой перезапуска `unless-stopped`. Порт 5433 используется для обеспечения совместимости с окружениями, где порт 5432 уже занят.

Сервис `backend` собирается из `backend/Dockerfile` и запускает FastAPI-приложение через `uvicorn`. Зависимость `depends_on` с условием `service_healthy` гарантирует, что бэкэнд не стартует до полной готовности базы данных. Параметр `extra_hosts: host.docker.internal:host-gateway` обеспечивает доступность хостовых СУБД из контейнера. `backend/Dockerfile` устанавливает системные пакеты `postgresql-client` и `mariadb-client`, что обеспечивает доступность CLI-утилит `pg_dump/pg_restore` и `mariadb-dump/mariadb` внутри контейнера для нативной стратегии управления состоянием баз данных.

Сервис `frontend` собирается из `frontend/Dockerfile` через многостадийную сборку: на первой стадии выполняется `next build` в `standalone`-режиме, на второй — копируется минимальный набор артефактов. Переменные `NEXT_PUBLIC_API_URL` и `NEXT_PUBLIC_WS_URL` задаются на этапе сборки и указывают на бэкэнд по адресу `localhost:8000`. Все три сервиса объединены в пользовательскую сеть `loadtest-network` типа `bridge`.

Тестовые СУБД развёртываются отдельно через собственные Docker Compose-файлы в директориях `../sakila/` и `../pagila/` — соседних относительно корня репозитория. MySQL с схемой Sakila и PostgreSQL с схемой Pagila при первом запуске автоматически инициализируют схему. Оба контейнера доступны из бэкенда через хост `host.docker.internal`.

3.4.1 Инициализация и миграции

Процедура старта бэкенда включает миграции Alembic и LogicalScenarioBootstrap; она описана выше при рассмотрении менеджера контекста lifespan. Цепочка Alembic включает 17 версионизуемых миграций, последовательно развивающих модель данных: от базовых таблиц прогонов, результатов и подключений — к индексам сценариев, профилям схем, группам баз данных и транзакционным наборам OLTP. Повторные запуски не создают дублирующих записей встроенных сущностей.

Конфигурация системы задаётся через файл `.env`: переменная `HISTORY_DATABASE_URL` задаёт подключение к базе данных проекта, `ENCRYPTION_KEY` — ключ шифрования паролей подключений, `API_HOST` и `API_PORT` — настройки API-сервера, `NEXT_PUBLIC_WS_URL` и `NEXT_PUBLIC_API_URL` — адреса для фронтенда. Файл `.env` не включается в систему контроля версий; вместо него поставляется шаблон `env.example`.

Полный запуск системы сводится к одной команде: `docker compose up -d --build`. После её выполнения поднимаются три сервиса из таблицы 3; при старте бэкенда автоматически выполняются шаги lifespan. Веб-интерфейс доступен по адресу `http://localhost:3000`, документация API — по адресу `http://localhost:8000/docs`.

Описанная программная реализация охватывает все компоненты, спроектированные в предыдущей главе, и обеспечивает полную функциональность системы нагрузочного тестирования.

3.5 Выводы по главе

В настоящей главе представлена программная реализация системы нагрузочного тестирования. Серверная часть разработана на Python 3.11 с использованием FastAPI, асинхронных драйверов СУБД и SQLAlchemy 2.0;

управление схемой автоматизировано цепочкой из 17 миграций Alembic. Реализованы пять репозитория доступа к данным, модуль генерации нагрузки с поддержкой одиночных SQL-запросов и транзакционных последовательностей операций, слой диалектов СУБД для унифицированного сбора внутренних метрик, а также модуль сравнительного анализа с двумя режимами работы и полным набором статистических критериев. Клиентская часть на основе Next.js 16 / React 19 предоставляет восемь страниц интерфейса с визуализацией метрик в реальном времени через WebSocket, Recharts и Plotly. Развёртывание реализовано через Docker Compose с автоматической инициализацией и идиоматичным bootstrap. На основе реализованной системы в следующей главе проводится экспериментальная проверка и анализ результатов.

4 ТЕСТИРОВАНИЕ И АНАЛИЗ РЕЗУЛЬТАТОВ

4.1 Методика тестирования и тестовое окружение

Тестирование разработанной системы проводилось на двух независимых наборах тестовых данных, охватывающих различные предметные области. Первый набор основан на схеме Sakila [11] — эталонной демонстрационной базе данных, разработанной командой MySQL. Второй — Brazilian E-Commerce Public Dataset [12], публичный набор данных электронной торговли с платформы Kaggle, содержащий сведения о реальных заказах.

Нагрузочные прогоны и эксплуатация разработанной системы выполнялись локально на одной рабочей станции без выделенного кластера и без распределения компонентов по отдельным серверам. Генерация нагрузки, сбор метрик, веб-интерфейс, служебная база данных проекта и все сравниваемые экземпляры СУБД размещались на общем вычислительном узле, что обеспечивало сопоставимость результатов при попарном сравнении движков в рамках одного прогона. Сравнение СУБД в таких условиях отражает относительное поведение при конкуренции за одни и те же ресурсы центрального процессора, оперативной памяти и подсистемы ввода-вывода, а не абсолютные показатели промышленного стенда.

Эксперименты выполнялись на персональном компьютере с процессором Intel Core i7-12700H двенадцатого поколения и архитектурой x86_64. На уровне хостовой операционной системы Windows рабочая станция оснащена 40 Гбайтами оперативной памяти. Непосредственно прогонам подлежала гостевая среда Ubuntu 24.04 LTS в подсистеме Windows для Linux второго поколения: для неё доступны десять физических ядер, двадцать логических процессоров и около 19 Гбайт оперативной памяти, поскольку WSL2 выделяет виртуальной машине часть ресурсов хоста, а не весь объём установленной памяти. Данные и контейнеры размещались на виртуальном дисковом томе объёмом около 1 Тбайт, предоставленном той же подсистемой.

На описанном стенде каждый набор данных был развёрнут в трёх экземплярах СУБД: MySQL 8, PostgreSQL 15 и MariaDB 10.x — в Docker-контейнерах. Серверная часть системы запускалась посредством Uvicorn, клиентская — через Next.js-сервер. База данных системы развёрнута

в контейнере PostgreSQL 15 на порту 5433. Для схемы Sakila подключения работали на портах 3306 для MySQL, 5437 для PostgreSQL и 3307 для MariaDB. Для набора Brazilian E-Commerce использовались порты 3308 для MySQL, 5434 для PostgreSQL и 3309 для MariaDB.

Для совместного тестирования трёх экземпляров СУБД как единого объекта применялась функция группы баз данных — именованной группы физических подключений под единым профилем схемы. Система автоматически определила профиль `sakila_like` для схемы Sakila и `olist_like` для набора Brazilian E-Commerce, после чего сформировала наборы сценариев нагрузки, специфичные для каждого профиля. В экспериментах использованы три из шести встроенных шаблонов: смешанная лёгкая нагрузка, только чтение и OLTP; остальные шаблоны каталога — только запись, смешанная тяжёлая нагрузка и аналитические запросы — в прогонах не задействовались.

Тестирование выполнялось в двух режимах системы: `per_test` и `series`. В режиме `per_test` все три подключения группы тестировались одновременно при фиксированных параметрах нагрузки; результаты использовались для попарного сравнения СУБД. В режиме `series` выполнялось по три прогона с различными уровнями нагрузки на каждый логический набор для анализа масштабируемости. Параметры нагрузки задавались числом виртуальных пользователей и числом итераций на пользователя; время прогрева во всех прогонах составляло 5 секунд. Во всех двенадцати прогонах параметр `use_indexes` был отключён: нагрузка выполнялась на базовой схеме данных без дополнительных индексов из спецификации набора сценария. Всего выполнено двенадцать прогонов по матрице демонстрационных экспериментов. На всём протяжении прогонов система фиксировала полный набор метрик, описанный в главе 2: время отклика с перцентилями и стандартным отклонением, пропускную способность, системные показатели хоста и внутренние метрики СУБД. Статистическая обработка результатов и формирование аналитических отчётов выполнялись встроенным модулем сравнительного анализа по алгоритму, описанному при проектировании и реализации в главах 2 и 3: проверка нормальности, критерии Уэлча и Манна–Уитни, размер эффекта Коэна d , поправка Бенджамини–Хохберга при множественных сравнениях. В таблицах и тексте ниже для сопоставления движков использованы

преимущественно средняя задержка, хвостовые перцентили и успешная пропускная способность; для сценариев с одиночными SQL-запросами она приведена в запросах в секунду, для OLTP-прогонов — в транзакциях в секунду.

Сравнительные характеристики развёртываний приведены в таблице 4.

Таблица 4 – Характеристики тестовых наборов данных

Логический набор	СУБД	Экземпляр	Профиль схемы	Таблиц	Тип первичного ключа
Sakila	MySQL	Sakila	sakila_ like	16	INT
Sakila	PostgreSQL	Pagila	sakila_ like	16	INT
Sakila	MariaDB	Makila	sakila_ like	16	INT
Brazilian E-Commerce	MySQL	Brazilian E-com	olist_ like	9	CHAR(32)
Brazilian E-Commerce	PostgreSQL	Brazilian E-com	olist_ like	9	CHAR(32)
Brazilian E-Commerce	MariaDB	Brazilian E-com	olist_ like	9	CHAR(32)

Логический набор Sakila, включающий экземпляры Sakila, Pagila и Makila на трёх СУБД, содержит 16 таблиц с целочисленными первичными ключами; Brazilian E-Commerce — 9 таблиц с ключами CHAR(32) и около 1,5 млн строк в совокупности. Различия в объёме и типе ключей позволяют сопоставить поведение СУБД в условиях лёгкой смешанной нагрузки и тяжёлых операций чтения с JOIN.

4.2 Тестирование на наборе данных Sakila

4.2.1 Сравнение СУБД при смешанной нагрузке

Прогоны на наборе Sakila в режиме `per_test` выполнялись со сценарием смешанной лёгкой нагрузки при 20 и 50 виртуальных пользователях, по 100 итераций каждый. Автоматически сгенерированный набор сценария нагрузки для профиля `sakila_like` включал операции на таблицах `rental` и `payment`: обновление временной метки в `rental`, вставку строки в `payment`, выборки по первичному и внешнему ключам, а также JOIN-запрос `rental` с таблицей `customer`.

Метрики производительности для каждой СУБД приведены в таблице 5.

Таблица 5 – Метрики производительности: Sakila, смешанная лёгкая нагрузка

Нагрузка	СУБД	p99, мс	Запросов/с
20 VU / 100 ит.	PostgreSQL	88	608
20 VU / 100 ит.	MySQL	56	602
20 VU / 100 ит.	MariaDB	38	596
50 VU / 100 ит.	PostgreSQL	125	610
50 VU / 100 ит.	MySQL	178	500
50 VU / 100 ит.	MariaDB	158	558

При умеренной нагрузке, 20 VU, все три СУБД демонстрируют близкие средние задержки и пропускную способность — около 27 мс и 600 запросов/с соответственно. Различия между СУБД выражены прежде всего в хвосте распределения задержек: наилучший p99 у MariaDB — 38 мс, тогда как у PostgreSQL и MySQL — 88 и 56 мс.

При высокой нагрузке, 50 VU, ситуация меняется: лидером по пропускной способности становится PostgreSQL — около 610 запросов/с против 500 у MySQL и 558 у MariaDB. MySQL при этом показывает наибольший p99 — около 178 мс, что указывает на рост редких медленных запросов при увеличении конкуренции. Попарный статистический анализ подтвердил значимость различий между PostgreSQL и MySQL по пропускной способности; различия между MySQL и MariaDB признаны незначимыми.

4.2.2 Масштабируемость при росте нагрузки

Для анализа масштабируемости на наборе Sakila в режиме `series` выполнены три прогона сценария только чтения с уровнями нагрузки 10, 20 и 40 виртуальных пользователей при 80 итерациях каждый. Набор сценария нагрузки включал шесть SELECT-операций: выборку строки по первичному ключу, выборку нескольких столбцов, выборку по внешнему ключу, JOIN с одной таблицей, JOIN с двумя таблицами и агрегацию `COUNT (*)`.

Серийный анализ Sakila в режиме `series` показывает закономерное нарастание хвостовой задержки: p95 у всех трёх СУБД растёт приблизительно на 65–72% при каждом переходе с 10 на 20 и с 20 на 40 виртуальных пользователей — типичный эффект очередей при увеличении конкуренции. Пропускная способность при этом также нарастает с уровнем нагрузки; на максимальном уровне, 40 VU, PostgreSQL сохраняет лидерство — около 585 запросов/с, MariaDB — около 380, MySQL — около 370. Рост хвоста отражает ожидаемую деградацию под нагрузкой, а не сбой СУБД.

4.3 Тестирование на наборе Brazilian E-Commerce

4.3.1 Сравнение СУБД при смешанной нагрузке

Прогоны на наборе Brazilian E-Commerce в режиме `per_test` выполнялись при тех же параметрах — 20 и 50 VU по 100 итераций, сценарий смешанной лёгкой нагрузки. Набор сценария нагрузки для профиля `olist_like` включал операции на таблицах `olist_customers_dataset`, `olist_geolocation_dataset`, `olist_order_items_dataset` и `olist_orders_dataset`: инкремент поля и выборку по первичному ключу, вставку строки, выборку по внешнему ключу и JOIN через FK-связи.

Метрики производительности для каждой СУБД приведены в таблице 6.

Таблица 6 – Метрики производительности: Brazilian E-Commerce, смешанная лёгкая нагрузка

Нагрузка	СУБД	Запросов/с
20 VU / 100 ит.	PostgreSQL	699
20 VU / 100 ит.	MySQL	427
20 VU / 100 ит.	MariaDB	639
50 VU / 100 ит.	PostgreSQL	681
50 VU / 100 ит.	MySQL	561
50 VU / 100 ит.	MariaDB	567

На наборе Brazilian E-Commerce лидерство PostgreSQL по пропускной способности значительно более выражено, чем на Sakila: при 20 VU — около 699 запросов/с против 427 у MySQL и 639 у MariaDB; при 50 VU — около 681 запросов/с против 561 и 567 соответственно. Это согласуется с более эффективной обработкой PostgreSQL тяжёлых JOIN по строковым ключам CHAR(32) на больших объёмах данных.

4.3.2 Масштабируемость при росте нагрузки

Для набора Brazilian E-Commerce в режиме `series` выполнены три прогона сценария только чтения на уровнях нагрузки 10, 20 и 40 виртуальных пользователей при 80 итерациях каждый. Набор сценария нагрузки включал шесть SELECT-операций на тех же таблицах с JOIN-запросами через FK-связи и агрегацией.

Серийный анализ в режиме `series` на наборе Brazilian E-Commerce демонстрирует иное поведение по сравнению с Sakila. Уже при 10 VU средняя задержка исчисляется сотнями миллисекунд, а пропускная способность всех трёх СУБД выходит на плато около 6 запросов/с и при росте числа виртуальных пользователей до 40 практически не увеличивается. Это объясняется тяжёлыми SELECT с JOIN по ключам CHAR(32) на объёме около 1,5 млн строк, а не отказом СУБД: насыщение достигается пределом самой рабочей нагрузки. На всех уровнях нагрузки PostgreSQL сохраняет наименьшую среднюю задержку.

4.4 Транзакционная нагрузка и верификация резервного копирования

Для проверки механизма резервного копирования и восстановления данных проведены два прогона транзакционной нагрузки — по одному на каждом наборе данных. Оба прогона выполнялись при 15 виртуальных пользователях и 50 итерациях в режиме `workload_mode=transaction`. В качестве сценария нагрузки использовался вручную сконфигурированный набор OLTP-транзакций: каждый виртуальный пользователь выполнял атомарные последовательности операций изменения данных, а не одиночные SQL-запросы; основной показатель пропускной способности в таких прогонах — успешные транзакции в секунду, дополнительно фиксировалась интенсивность SQL-шагов (запросов в секунду). Поскольку сценарий содержит write-операции, система автоматически активировала механизм резервного копирования.

Для набора Sakila набор транзакций включал операции с таблицами `payment` и `rental`: выборку и обновление записей по первичному ключу, вставку новой записи и откат при нарушении ограничений. Для набора Brazilian E-Commerce — операции с таблицами `olist_customers_dataset` и `olist_orders_dataset`: выборку, обновление статуса заказа и вставку записи о событии.

Перед началом каждого прогона система зафиксировала снимок состояния тестируемых баз данных. По завершении теста выполнила восстановление и верификацию целостности путём сравнения числа строк в каждой таблице с зафиксированными значениями. По результатам обоих прогонов счётчики строк во всех таблицах совпали с исходными значениями, расхождений не обнаружено.

4.5 Сопоставление с внешним генератором нагрузки Sysbench

Для проверки корректности учёта метрик на наборе Sakila выполнена контрольная серия с внешним генератором Sysbench при согласованных параметрах: 30 виртуальных пользователей, 100 итераций, прогрев 10 секунд, по 3000 операций на каждую из трёх СУБД. Нагрузка задавалась зеркальными Lua-сценариями для сценариев `oltp` и `mixed_light`, а также фиксированным запросом `SELECT * FROM rental WHERE rental_id`

= 1 в режиме пользовательского SQL.

Во всех прогонах разработанной системы зафиксировано по 3000 успешных операций на каждую СУБД без ошибок выполнения; встроенная самопроверка метрик замечаний не выдала. Абсолютная пропускная способность в системе ниже, чем в отчётах Sysbench, что объясняется полным клиентским и измерительным стеком и не указывает на ошибку в расчёте показателей. Сопоставление с Sysbench подтвердило воспроизводимость объёма нагрузки и корректность учёта.

4.6 Выводы по главе

Экспериментальная проверка в двенадцати прогонах на локальном одноузловом стенде, на двух логических наборах данных и трёх СУБД подтвердила работоспособность разработанной системы: группы баз данных и профили схем, режимы `per_test` и `series`, сравнительный анализ с отчётами и резервное копирование при OLTP-нагрузке. Контрольная серия на наборе Sakila с внешним генератором Sysbench подтвердила корректность учёта объёма нагрузки и метрик при согласованных сценариях; расхождение абсолютной пропускной способности с краткими отчётами Sysbench укладывается в ожидаемую разницу клиентских стеков. По результатам сравнения лидерство СУБД зависит от профиля схемы и уровня нагрузки: на компактной схеме Sakila при умеренной конкуренции различия проявляются прежде всего в хвосте задержек, при высокой — в пользу PostgreSQL по пропускной способности; на тяжёлом наборе Brazilian E-Commerce PostgreSQL устойчиво опережает конкурентов, а серийный анализ фиксирует насыщение рабочей нагрузки, а не отказ движков. Все прогоны завершены без ошибок выполнения запросов; write-сценарии воспроизводимы за счёт верифицированного восстановления состояния БД. Полученные данные демонстрируют практическую ценность системы как инструмента объективного сравнения реляционных СУБД в контролируемых условиях.

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы был проведён анализ ограничений существующих инструментов нагрузочного тестирования реляционных СУБД, выявлен функциональный разрыв — отсутствие открытого решения, совмещающего генерацию нагрузки на нескольких СУБД, мониторинг метрик в реальном времени, централизованное хранение истории и автоматизированный сравнительный анализ, — и сформулированы требования к разрабатываемой системе. На их основе спроектирована и реализована многоуровневая клиент-серверная система сравнительного нагрузочного тестирования реляционных баз данных с веб-интерфейсом.

Ключевым техническим результатом работы является ядро генерации нагрузки, реализующее параллельное асинхронное выполнение как одиночных параметризованных SQL-запросов, так и транзакционных последовательностей операций с заданным числом виртуальных пользователей, двухфазным прогревом и отдельной фазой измерения. Сбор метрик охватывает четыре группы показателей — время отклика с полной описательной статистикой, пропускную способность с учётом режима нагрузки, системные метрики хоста и внутренние метрики СУБД — и организован через унифицированный слой диалектов, что обеспечивает единообразный набор показателей для MySQL, PostgreSQL и MariaDB без изменений в основном цикле тестирования. Трёхуровневый механизм управления сценариями — шаблоны, профили схем и наборы сценариев нагрузки — позволяет автоматически подбирать SQL-конфигурацию под любую поддерживаемую схему данных и тестировать группу разнородных СУБД в рамках одного прогона. Модуль резервного копирования и восстановления с верификацией целостности обеспечивает воспроизводимость прогонов с операциями записи. Статистический анализ реализован как конвейер: проверка нормальности по критерию Шапиро–Уилка, выбор критерия сравнения — Уэлча или Манна–Уитни, — вычисление размера эффекта Коэна d и поправка на множественные сравнения методом Бенджамини–Хохберга.

Экспериментальная проверка проведена в 12 прогонах на двух тестовых наборах данных — Sakila и Brazilian E-Commerce — с тремя СУБД. Результаты показали, что поведение СУБД существенно зависит как от

характера данных, так и от уровня нагрузки. На наборе Sakila с целочисленными ключами при умеренной конкуренции три СУБД практически неразличимы по средней задержке, а при росте числа виртуальных пользователей до 50 PostgreSQL выходит в лидеры по пропускной способности. На наборе Brazilian E-Commerce со строковыми первичными ключами CHAR(32) PostgreSQL устойчиво опережает конкурентов на обоих уровнях нагрузки, что обусловлено более эффективным планировщиком запросов при работе с такими ключами. Серийный анализ выявил ожидаемый рост p95 на 65–72% при увеличении нагрузки на Sakila, тогда как на Brazilian E-Commerce зафиксировано насыщение рабочей нагрузки при неизменной пропускной способности — характерный паттерн сценария `read_only` с тяжёлыми JOIN-операциями. Доля ошибок во всех 12 прогонах составила 0%; механизм резервного копирования подтвердил корректность восстановления данных на обоих наборах.

Полученные результаты демонстрируют, что выбор СУБД не является универсальным: преимущество одной системы над другой зависит от структуры данных, типа запросов и уровня конкурентности. Именно для выявления таких зависимостей в контролируемых условиях и предназначена разработанная система. Практическая значимость работы состоит в создании инструмента, позволяющего инженерам по производительности и разработчикам получать статистически обоснованные сравнительные оценки без ручной обработки данных и без установки специализированного программного обеспечения.

Перспективными направлениями развития системы являются поддержка NewSQL-СУБД и дополнительных реляционных СУБД, горизонтальное масштабирование генераторов нагрузки, автоматический подбор параметров прогрева, а также расширение набора статистических методов — в частности, анализ временных трендов и обнаружение точек изменения характеристик производительности. Отдельным направлением является интеграция системы в процессы непрерывной интеграции и поставки: наличие REST API создаёт техническую основу для автоматического запуска прогонов при изменениях кода и для сравнения результатов с эталонными значениями предыдущих версий, что позволит обнаруживать регрессии производительности до их попадания в продуктивную среду.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Gregg B. Systems Performance: Enterprise and the Cloud. — 2-е изд. — Hoboken, NJ : Addison-Wesley Professional, 2020. — ISBN 978-0-13-682009-2.
2. Kleppmann M. Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. — Sebastopol, CA : O'Reilly Media, 2017. — ISBN 978-1-4493-7332-0.
3. Transaction Processing Performance Council. TPC Benchmark Specifications. — URL: https://www.tpc.org/tpc_documents_current_versions/current_specifications5.asp (дата обращения 14.01.2026).
4. The Linux Foundation. Prometheus Overview. — URL: <https://prometheus.io/docs/introduction/overview/> (дата обращения 18.01.2026).
5. Cloud Native Computing Foundation. OpenTelemetry Documentation. — URL: <https://opentelemetry.io/docs/> (дата обращения 19.01.2026).
6. TPC Council. HammerDB v5.0 Release Notes. — URL: <https://www.hammerdb.com/docs/ch01s01.html> (дата обращения 19.01.2026).
7. Quest Software. Benchmark Factory for Database 9.1 User Guide. — URL: <https://support.quest.com/technical-documents/benchmark-factory-for-database/9.1/user-guide> (дата обращения 19.01.2026).
8. Exasol AG. Benchkit: A Framework for Reproducible Database Benchmarking. — URL: <https://github.com/exasol/benchkit> (дата обращения 20.01.2026).
9. Ramírez S. FastAPI Documentation. — URL: <https://fastapi.tiangolo.com/> (дата обращения 25.01.2026).
10. The PostgreSQL Global Development Group. PostgreSQL 15 Documentation. — URL: <https://www.postgresql.org/docs/15/> (дата обращения 03.02.2026).
11. Oracle Corporation. The Sakila Sample Database. — URL: <https://dev.mysql.com/doc/sakila/en/> (дата обращения 21.01.2026).
12. Olist. Brazilian E-Commerce Public Dataset by Olist. — URL: <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce> (дата обращения 12.03.2026).

ПРИЛОЖЕНИЕ А

Ссылка на GitHub

Перейти в репозиторий можно по QR-коду на рисунке А.1



Рисунок А.1 – QR-код на репозиторий

ПРИЛОЖЕНИЕ Б

Использованные датасеты

На рисунке Б.1 представлена схема данных Sakila/Pagila:

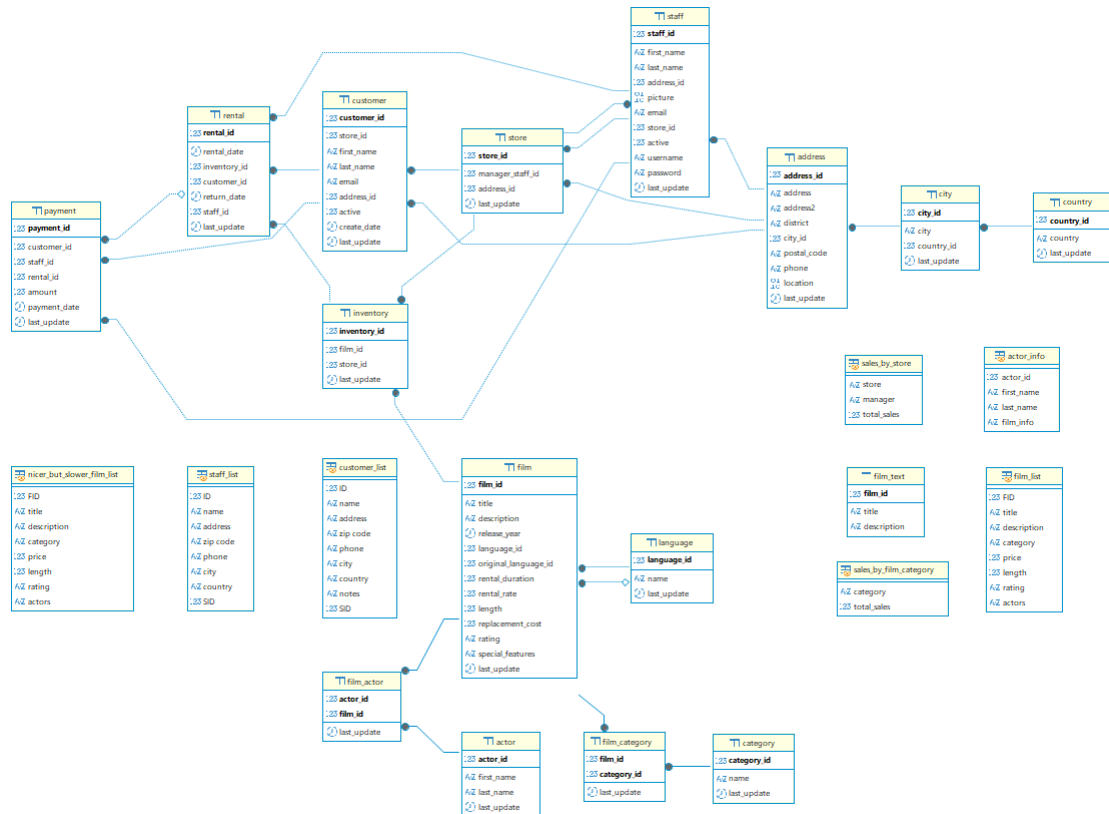


Рисунок Б.1 – ER-диаграмма Sakila/Pagila

На рисунке Б.2 представлена схема данных Brazilian E-Commerce Public Dataset by Olist:

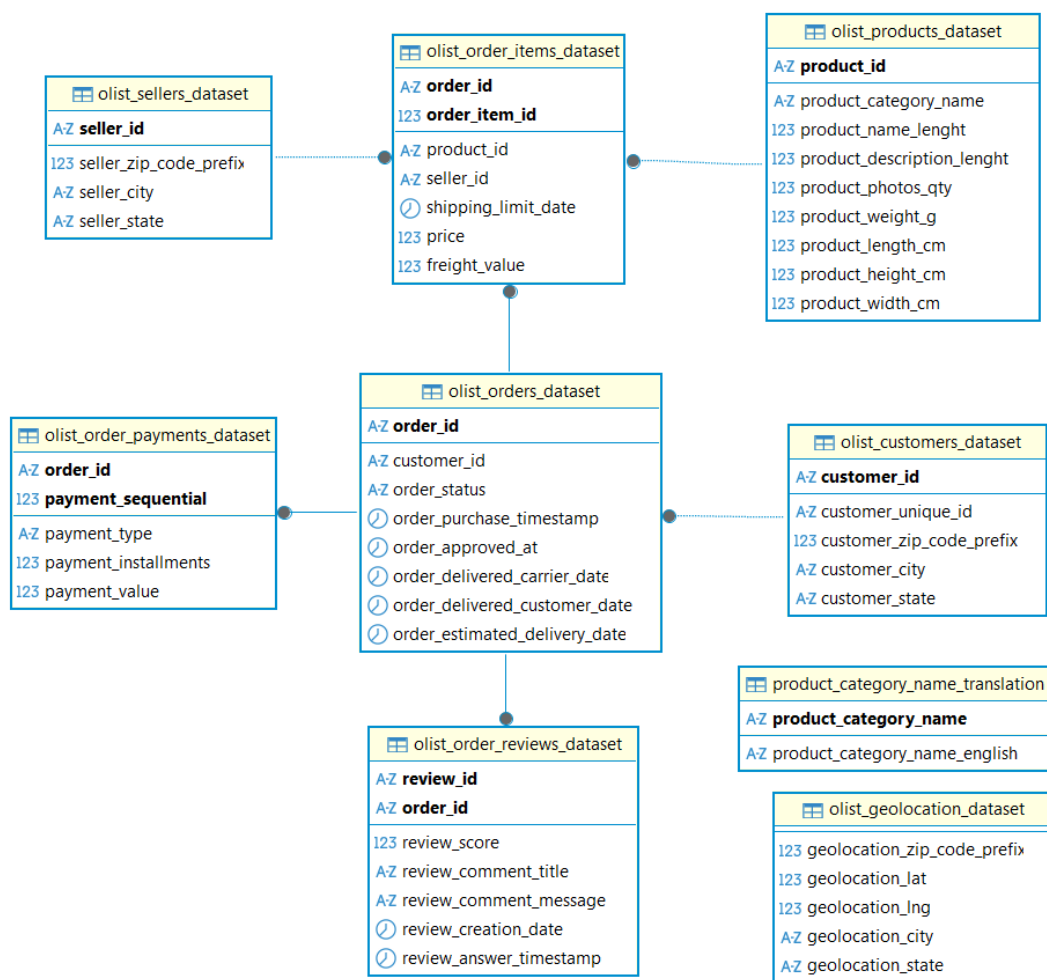


Рисунок Б.2 – ER-диаграмма Brazilian E-Commerce Public Dataset by Olist